UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
DISCIPLINA: ENGENHARIA DE SOFTWARE
PROFESSOR: ARTURO HERNANDEZ DOMINGUEZ

Um Sistema de Gerenciamento de Estágios Universitários Usando Microservices e Componentes de Software [e AgentesAI]

Victor Moura Amaral de Oliveira
Lorenzo Holanda

Repositório GitHub: https://github.com/victorMAOliveira/projeto-eng-soft

Maceió
2026

Sumário

1 Enunciado
- 1.1 Problema
- 1.2 Objetivo
- 1.3 Escopo
- 1.4 Atores
- 1.5 Requisitos funcionais
- 1.6 Requisitos não funcionais

Parte 1 — Modelagem

2 Metodologia
- 2.1 Escolha: Scrum
- 2.2 Papéis
- 2.3 Cerimônias e artefatos
- 2.4 Duração das sprints

3 Técnicas de reuso
- 3.1 Microservices
- 3.2 Componentes de Software

4 Desenvolvimento
- 4.1 Narrativa por sprint
- 4.2 Catálogo de componentes
- 4.3 Matriz de reuso
- 4.4 Métricas de reuso
- 4.5 Qualidade: testes
- 4.6 Padrões de microsserviços adotados
- 4.7 Resumo dos ADRs

5 Backlogs
- 5.1 Product Backlog
- 5.2 Sprint Backlogs

6 Projeto da solução
- 6.1 Arquitetura
- 6.2 API Gateway
- 6.3 Serviços e componentes
- 6.4 Recomendação
- 6.5 Serviços em execução

Parte 2 — Implementação

7 Tecnologias
- 7.1 Microsserviços
- 7.2 Infraestrutura e bibliotecas

8 Funcionamento do sistema
9 Repositório
10 Referências
11 Vídeo
Apêndice A — Checklist de conformidade

1 Enunciado

1.1 Problema

O estágio supervisionado é regido pela Lei nº 11.788/2008 (BRASIL, 2008), que impõe regras objetivas: jornada de até 6 horas diárias e 30 semanais para o ensino superior, duração de no máximo 2 anos na mesma concedente (exceto para estagiário com deficiência), relatório de atividades em periodicidade não superior a 6 meses e seguro contra acidentes pessoais como condição do Termo de Compromisso de Estágio (TCE). Na prática, esse ciclo — vaga, candidatura, seleção, termo, aprovação da coordenação e acompanhamento — costuma ser conduzido por e-mail e planilhas, o que dificulta encontrar vagas compatíveis, controlar prazos e garantir o cumprimento da lei.

1.2 Objetivo

Construir o **EstágioHub**, um sistema web que gerencia o ciclo de vida do estágio universitário de ponta a ponta, aplicando duas técnicas de reuso — **microsserviços** e **componentes de software** — e, opcionalmente, **agentes de IA**. O sistema deve validar automaticamente as regras legais, gerar o TCE em PDF, recomendar vagas com explicação e notificar cada ator a cada transição.

1.3 Escopo

Incluído: cadastro e login com papéis; perfis acadêmico e da empresa; publicação e busca de vagas; recomendação híbrida explicável; candidatura com máquina de estados; ranking de candidatos; geração e aprovação do TCE; relatórios de atividades com versões; notificações; agentes de IA (assistente de candidatura, acompanhamento de prazos e triagem); painel de status dos microsserviços. Fora do escopo: assinatura digital com validade jurídica, integração com sistemas acadêmicos institucionais e pagamentos de bolsa (história US-31, classificada como *Won't*).

1.4 Atores

Tabela 1 — Atores do sistema.

Ator	Responsabilidades no sistema
Estudante	Mantém o perfil acadêmico (curso, período, habilidades, resumo, disponibilidade, modalidade), busca vagas, recebe recomendações explicadas, candidata-se, acompanha o TCE e envia relatórios.
Empresa concedente	Publica vagas, recebe o ranking de candidatos e aprova ou rejeita candidaturas, informando os termos do estágio.
Coordenação de estágio	Aprova os termos de compromisso, acompanha relatórios e recebe alertas de prazos.

1.5 Requisitos funcionais

Tabela 2 — Requisitos funcionais.

ID	Requisito	Histórias
RF01	Cadastrar e autenticar usuários com papéis (estudante, empresa, coordenação).	US-06
RF02	Manter perfil acadêmico e perfil da concedente.	US-07, US-08
RF03	Publicar, buscar, filtrar e encerrar vagas, validando a jornada legal.	US-09–US-11
RF04	Recomendar vagas ao estudante com explicação do porquê; *cold start* sem perfil.	US-20
RF05	Candidatar-se; analisar, aprovar, rejeitar, desistir, concluir e rescindir pela máquina de estados (transição inválida = HTTP 409).	US-12, US-13
RF06	Validar jornada (6 h/30 h), duração de 2 anos (exceto PcD), seguro do TCE e prazo semestral de relatório.	US-14
RF07	Gerar o TCE em PDF automaticamente e permitir sua aprovação pela coordenação, ativando o estágio.	US-16, US-17
RF08	Enviar relatórios de atividades com versões.	US-18
RF09	Notificar os atores envolvidos a cada transição.	US-19
RF10	Exibir o ranking de candidatos de uma vaga.	US-21
RF11	Oferecer agentes de IA (assistente, acompanhamento e triagem) com fallback em regras.	US-23–US-25
RF12	Exibir o status de todos os microsserviços e destacar na interface quais serviços forneceram cada dado.	US-26, US-27

1.6 Requisitos não funcionais

Tabela 3 — Requisitos não funcionais.

ID	Requisito	Como é atendido
RNF01	Arquitetura de microsserviços com um banco por serviço	8 serviços + frontend, 7 bancos isolados (seção 6.1)
RNF02	Reuso por componentes instaláveis	6 pacotes em `components/` com versão semântica e cobertura ≥ 80% (seção 4)

ID	Requisito	Como é atendido
RNF03	Segurança	argon2id, JWT com expiração, papéis no gateway e nos serviços, validação pydantic, CORS restrito, containers não-root
RNF04	Resiliência e degradação graciosa	*timeout*, *retry*, *circuit breaker*, DLQ, idempotência; a interface segue funcionando sem recomendação/agentes
RNF05	Observabilidade	log JSON com *correlation-id*, `/health`, `/ready`, `/version`, `GET /services`
RNF06	Execução reprodutível	`make up` sobe tudo via Docker Compose; ferramentas em containers
RNF07	Qualidade	`ruff`, testes unitários/de API, e2e, CI no GitHub Actions
RNF08	Interface	responsiva, sem CDN (funciona sem internet), em português

Parte 1 — Modelagem

2 Metodologia

2.1 Escolha: Scrum

A equipe adotou o **Scrum** (SCHWABER; SUTHERLAND, 2020). A justificativa é que o projeto tinha escopo grande, prazo curto e um contrato de aceitação explícito (o modelo de relatório): o Scrum permite fatiar o produto em incrementos funcionais e verificáveis, priorizar pelo valor (MoSCoW) e inspecionar e adaptar a cada iteração — o que foi necessário, por exemplo, quando problemas técnicos reais (seção 4) exigiram correções em componentes já entregues. Por ser um trabalho intensivo, cada sprint foi uma **micro-sprint** de cerca de 2 horas, encerrada com um incremento integrado, testado e marcado no Git (tags `sprint-0` a `sprint-4`).

2.2 Papéis

Tabela 4 — Papéis Scrum.

Papel Scrum	Responsável
Product Owner	Victor Moura Amaral de Oliveira
Scrum Master	Lorenzo Holanda
Time de desenvolvimento	Todos os integrantes

2.3 Cerimônias e artefatos

- **Sprint Planning:** no início de cada micro-sprint, seleção das histórias do Product Backlog e quebra em tarefas técnicas estimadas (seção 5).
- **Daily Scrum:** substituída, no formato intensivo, por sincronizações curtas no início de cada bloco de trabalho. [PREENCHER: como a equipe conduziu as sincronizações]
- **Sprint Review:** demonstração do incremento contra a Definition of Done, com evidências (commits, tag, resultado de `make test`) registradas em `docs/scrum/sprint-N.md`.
- **Sprint Retrospective:** registrada em cada `sprint-N.md`. [PREENCHER: retrospectivas da equipe]
- **Artefatos:** Product Backlog (`docs/scrum/product-backlog.md`), Sprint Backlogs (`docs/scrum/sprint-0.md` a `sprint-4.md`), incremento (tag Git) e ADRs (`docs/architecture/adr/`).

Definition of Done global: código com *docstrings*; testes passando; `ruff` sem erros; serviço *healthy* no Compose; documentação atualizada.

2.4 Duração das sprints

O projeto ocorreu de 08/11/2026 a 09/11/2026, em cinco micro-sprints de cerca de 2 horas. O horário real de cada sprint corresponde aos *timestamps* do primeiro e do último commit dela (`git log sprint-N-1..sprint-N`).

Tabela 5 — Sprints.

Sprint	Meta	Horário (início–fim)
0 — Fundação	Repositório, componentes, Compose, gateway, CI, ADRs	[PREENCHER: horário real pelos commits]
1 — Identidade e vagas	`user-service`, `internship-service`, login e vagas na interface	[PREENCHER: horário real pelos commits]
2 — Candidaturas e documentos	`application-service`, `document-service`, eventos, notificações, saga	[PREENCHER: horário real pelos commits]
3 — Inteligência	`recommendation-service` + avaliação offline, `agent-service`	[PREENCHER: horário real pelos commits]
4 — Qualidade e entrega	Interface, e2e, prints, relatório	[PREENCHER: horário real pelos commits]

3 Técnicas de reuso

O projeto aplica duas técnicas de reuso — **Microservices** e **Componentes de Software** — e as combina: os microsserviços reutilizam componentes.

3.1 Microservices

Microsserviços são um estilo arquitetural em que uma aplicação é composta por **serviços pequenos, independentes e implantáveis separadamente**, cada um executando em seu próprio processo, organizado em torno de uma capacidade de negócio e comunicando-se por mecanismos leves, como HTTP e mensageria (LEWIS; FOWLER, 2014; NEWMAN, 2021). Cada serviço é dono dos seus dados (*Database per Service*) e pode evoluir, escalar e falhar de forma isolada (RICHARDSON, 2018). Como técnica de **reuso**, um microsserviço expõe uma capacidade por meio de uma interface estável (API REST ou eventos) que pode ser reutilizada por vários consumidores sem que eles conheçam sua implementação: no EstágioHub, por exemplo, o `recommendation-service` é reutilizado pelo frontend (página de recomendações e ranking) e pelo `agent-service` (ferramentas `get_recommendations` e `get_candidate_ranking`), e o `notification-service` reage a eventos de todos os demais sem que nenhum deles dependa dele.

Os padrões de microsserviços adotados (API Gateway, Database per Service, comunicação síncrona resiliente, mensageria, saga coreografada, CQRS leve, Health Check API) estão na seção 4.6.

3.2 Componentes de Software

Szyperski, Gruntz e Murer (2002) definem um componente de software como uma *unidade de composição com interfaces especificadas contratualmente e apenas dependências de contexto explícitas*, que pode ser implantada de forma independente e composta por terceiros. Sommerville (2018) apresenta a engenharia de software baseada em componentes (CBSE) como o reuso de componentes independentes, padronizados e acessados apenas por suas interfaces, separando a **interface fornecida** (serviços que o componente oferece) da **interface requerida** (o que ele precisa do ambiente).

No EstágioHub, cada componente em `components/` é um **pacote Python instalável** (`pyproject.toml` com versão semântica), com interface pública explícita no `__init__.py` (`__all__`), README documentando interfaces fornecida e requerida, exemplo de uso e lista de serviços que o reutilizam, e testes com cobertura mínima de 80%. Os componentes não dependem uns dos outros: quando precisam colaborar, recebem a dependência como parâmetro (por exemplo, `eh_http.ResilientClient` recebe um `header_provider` em vez de importar `eh_observability`), o que mantém as dependências de contexto explícitas, como pede a definição de Szyperski.

4 Desenvolvimento

4.1 Narrativa por sprint

Sprint 0 — Fundação. Foram criados os seis componentes (`eh_observability`, `eh_http`, `eh_auth`, `eh_db`, `eh_events` e um esqueleto do `eh_ui`), o `api-gateway`, o `docker-compose.yml` com RabbitMQ e gateway, o `Makefile`, a imagem `tools` (lint, testes, seed, e2e) e o CI. Aplicação das técnicas: o gateway já nasceu como composição de três componentes. Problemas reais: o host de desenvolvimento (Windows) não tinha GNU Make, o que levou a um `Makefile` que só chama `docker`/`docker compose` (compatível com `sh` e `cmd.exe`); e o proxy do gateway duplicava headers por diferença de maiúsculas/minúsculas, corrigido no `eh_http` com `httpx.Headers`.

Sprint 1 — Identidade e vagas. `user-service` (argon2, JWT, perfis, eventos `user.registered`/`profile.updated`), `internship-service` (validação legal da jornada também como *check constraint*) e o `web-frontend` (login, dashboard por papel, vagas, perfis). Problemas reais: `MissingGreenlet` ao ler `updated_at` após um UPDATE assíncrono — corrigido **no componente** `eh_db` (versão 0.1.1, `eager_defaults`), beneficiando todos os serviços de uma vez; cobertura subestimada por causa de *greenlets* (configuração `concurrency = ["thread", "greenlet"]`); log de acesso duplicado (corrigido no `eh_observability` 0.1.1); e a porta 8080 ocupada em IPv6 por um processo do WSL no host.

Sprint 2 — Candidaturas e documentos. `application-service` (máquina de estados e regras da Lei 11.788 em funções puras), `document-service` (TCE em PDF, aprovação, relatórios com versões), `notification-service` (caixas no Redis) e a saga coreografada, validada contra a stack real. Os contratos de eventos ganharam campos opcionais (`eh_events` 0.2.0, mudança compatível). Problema real: *lazy load* assíncrono do relacionamento `placement` em candidaturas recém-criadas.

Sprint 3 — Inteligência. `recommendation-service` (modelo de leitura por eventos, algoritmo híbrido explicável, *rebuild*), conjunto sintético rotulado e avaliação offline, e `agent-service` (três agentes, Claude com *tool use* e fallback em regras). Problemas reais: modelo de leitura vazio para dados anteriores ao serviço (resolvido pelo `rebuild`) e o prazo do primeiro relatório, que contava da aprovação e passou a contar do início do estágio.

Sprint 4 — Qualidade e entrega. Tela de status, legenda e destaque dos microsserviços, seed, teste e2e, prints automatizados com Playwright (que exercitam o fluxo principal pela própria interface) e este relatório. Problema real: a imagem oficial do Playwright para Python não traz o pacote `playwright`; foi criada uma imagem fina com a versão correspondente.

Os detalhes (tarefas, estimativas, evidências e problemas) estão nos arquivos `docs/scrum/sprint-N.md`.

4.2 Catálogo de componentes

Tabela 6 — Catálogo de componentes (versões lidas dos `pyproject.toml`).

Componente	Versão	Descrição	LOC	Serviços que reutilizam
`eh_auth`	1.0.0	JWT creation/validation, argon2 password hashing and FastAPI role guards.	161	8
`eh_db`	1.0.0	async SQLAlchemy 2 engine/session, base repository, Alembic helpers and SQL idempotency store.	164	5
`eh_events`	1.0.0	event contracts (pydantic), async RabbitMQ publisher/consumer with retry, idempotency and DLQ.	473	7
`eh_http`	1.0.0	resilient httpx client with timeout, retry/backoff and circuit breaker.	133	6
`eh_observability`	1.0.0	JSON logging, correlation-id middleware and health/ready/version routes.	213	9
`eh_ui`	1.0.0	reusable Jinja macros (cards, tables, service badge, alerts), filters and CSS.	376	1

4.3 Matriz de reuso

Tabela 7 — Matriz de reuso componente × serviço (calculada por análise estática dos imports).

Serviço	eh_auth	eh_events	eh_db	eh_observability	eh_http	eh_ui
agent-service	●	●	●	●	●	
api-gateway	●			●	●	
application-service	●	●	●	●	●	
document-service	●	●		●	●	
internship-service	●	●	●	●		

Serviço	eh_auth	eh_events	eh_db	eh_observability	eh_http	eh_ui
notification-service	●	●		●		
recommenda-tion-service	●	●	●	●	●	
user-service	●	●	●	●		
web-frontend				●	●	●

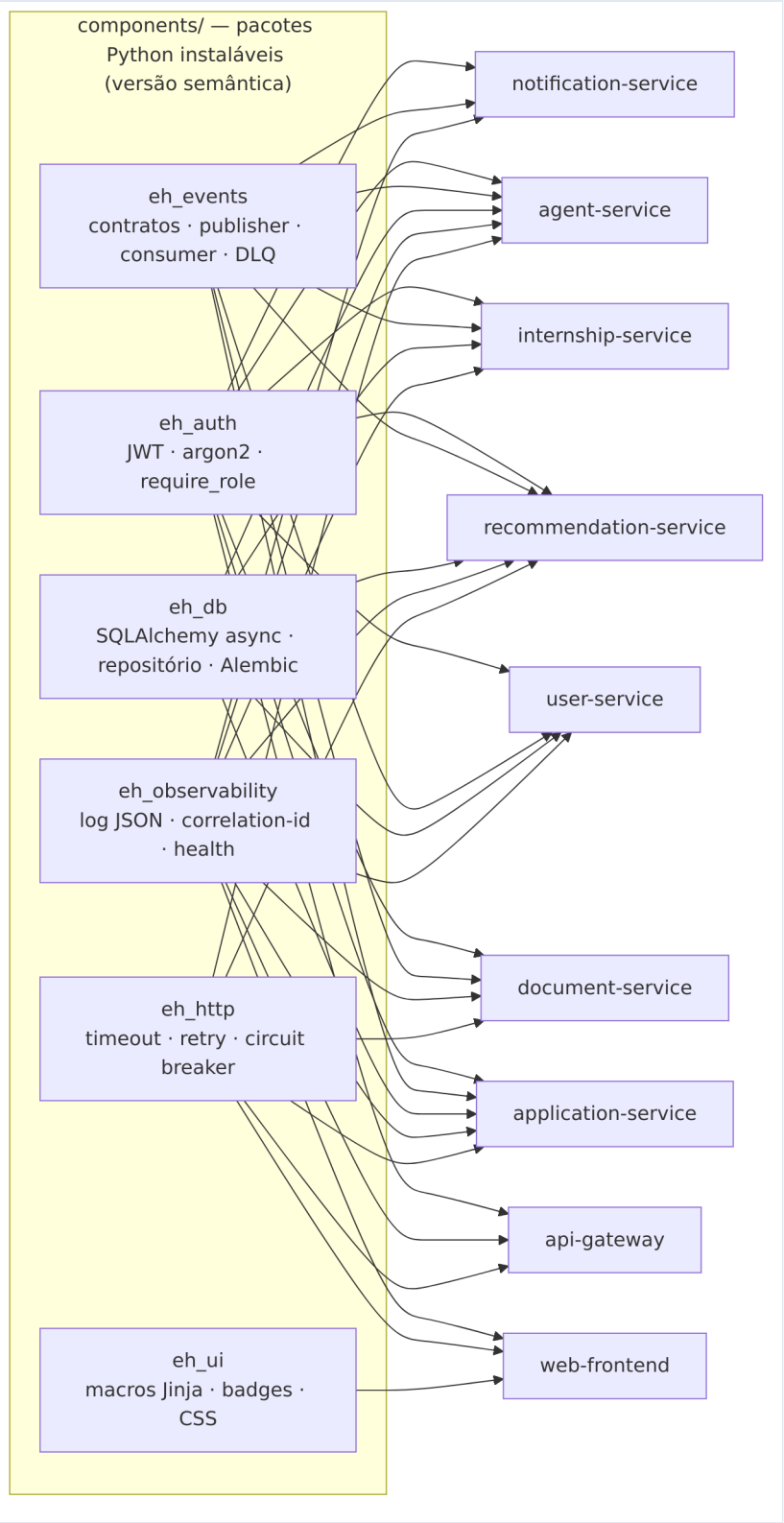


Figura 1 — Componentes reutilizados por cada serviço.

4.4 Métricas de reuso

As métricas são calculadas por `scripts/reuse_metrics.py` (executado por `make eval`): conta linhas lógicas (sem linhas em branco, comentários e *docstrings*) de cada serviço e dos componentes que ele importa.

Tabela 8 — Linhas reutilizadas dos componentes × linhas específicas de cada serviço.

Serviço	LOC específicas	LOC reutilizadas (componentes)	Componentes usados	Taxa de reuso
agent-service	1059	1144	5	51,9%
api-gateway	317	507	3	61,5%
application-service	982	1144	5	53,8%
document-service	807	980	4	54,8%
internship-service	322	1011	4	75,8%
notification-service	375	847	3	69,3%
recommendation-service	875	1144	5	56,7%
user-service	492	1011	4	67,3%
web-frontend	2320	722	3	23,7%
Total	**7549**	**8510**	—	**53,0%**

Sem componentes, o código comum seria copiado em cada serviço: 16059 LOC no total, contra 9069 LOC com componentes — **6990 LOC evitadas**.

4.5 Qualidade: testes

Tabela 9 — Resultado de `make test` (lint, testes unitários e de API, cobertura).

Tipo	Pacote	Testes	Cobertura	Resultado
lint	ruff	—	—	aprovado
componente	eh_auth	11	100%	aprovado
componente	eh_db	7	100%	aprovado
componente	eh_events	13	100%	aprovado
componente	eh_http	9	100%	aprovado
componente	eh_observability	10	100%	aprovado
componente	eh_ui	9	100%	aprovado
serviço	agent-service	11	95%	aprovado
serviço	api-gateway	11	98%	aprovado
serviço	application-service	114	97%	aprovado
serviço	document-service	12	92%	aprovado
serviço	internship-service	12	98%	aprovado
serviço	notification-service	15	95%	aprovado
serviço	recommendation-service	14	98%	aprovado
serviço	user-service	16	97%	aprovado
frontend	web-frontend	31	98%	aprovado
	Total	**295**		

Além disso, `make e2e` executa o fluxo principal completo via gateway contra a stack do Compose (seção 8).

4.6 Padrões de microsserviços adotados

Tabela 10 — Padrões adotados.

Padrão	Onde	Como
API Gateway	`api-gateway`	Ponto único de entrada em Python: rotas por prefixo, JWT, política de papéis, *correlation-id*, `GET /services` (seção 6.2).
Database per Service	todos	7 containers de banco; nenhum serviço acessa o banco de outro.
Comunicação síncrona resiliente	`eh_http`	REST com *timeout*, *retry* com *backoff* (só idempotentes, ou POST sem conexão) e *circuit breaker* (NYGARD, 2018).
Mensageria assíncrona	`eh_events`	*Topic exchange* `estagiohub.events`, filas duráveis, consumidor idempotente e *dead-letter queue* (HOHPE; WOOLF, 2003).
Saga coreografada	`application-service` + `document-service`	aprovação → TCE → aprovação da coordenação → estágio ativo, sem orquestrador (GARCIA-MOLINA; SALEM, 1987; RICHARDSON, 2018).
CQRS leve / read model	`recommendation-service`	Modelo de leitura próprio alimentado por eventos (FOWLER, 2011).
Event-carried state transfer	contratos	O *payload* leva os dados de que o consumidor precisa (ex.: termos do TCE).
Health Check API	`eh_observability`	`/health` (*liveness*), `/ready` (banco + broker) e `/version`; `depends_on: service_healthy` no Compose.
Degradação graciosa	gateway + frontend	Upstream indisponível vira 503 com o nome do serviço; o painel afetado mostra um aviso e os demais seguem funcionando.

4.7 Resumo dos ADRs

Tabela 11 — Architecture Decision Records (`docs/architecture/adr/`).

ADR	Decisão
001	Microsserviços com Database per Service.
002	Python 3.12 + FastAPI em todos os serviços.
003	API Gateway próprio em Python (não Kong/Traefik).
004	Eventos no RabbitMQ e saga coreografada.

ADR	Decisão
005	Reuso por componentes instaláveis com versão semântica.
006	Ferramentas em containers e host enxuto.
007	Frontend renderizado no servidor (Jinja2 + HTMX), JWT em *cookie* HttpOnly.
008	MongoDB para metadados de documentos e volume para arquivos.
009	Notificações em Redis, alimentadas por todos os eventos.
010	Recomendador híbrido, explicável e determinístico (sem LLM).
011	Agentes de IA com *tool use* e fallback em modo regras.

5 Backlogs

5.1 Product Backlog

Tabela 12 — Product Backlog (critérios de aceitação Given/When/Then em `docs/scrum/product-backlog.md`).

ID	História	MoSCoW	SP	Sprint
US-01	Como pessoa avaliadora, quero subir todo o sistema com um único comando, para avaliar o projeto sem configuração manual.	Must	5	0
US-02	Como equipe, quero componentes de software reutilizáveis e instaláveis, para não duplicar código entre os microsserviços.	Must	8	0
US-03	Como cliente da API, quero um ponto único de entrada que valide o JWT e o papel, para centralizar segurança e roteamento.	Must	8	0
US-04	Como equipe, quero CI com lint, testes e build, para detectar regressões a cada *push*.	Should	3	0
US-05	Como operador, quero `/health`, `/ready` e `/version` em todo serviço, para saber se está saudável.	Must	3	0
US-06	Como usuário, quero me cadastrar e fazer login com meu papel, para acessar as funções do meu perfil.	Must	5	1
US-07	Como estudante, quero manter meu perfil acadêmico, para receber recomendações melhores.	Must	3	1

ID	História	MoSCoW	SP	Sprint
US-08	Como empresa, quero manter o perfil da concedente, para que os estudantes a conheçam.	Should	2	1
US-09	Como empresa, quero publicar vaga com requisitos, curso, período mínimo, carga horária, modalidade e bolsa, para atrair estudantes.	Must	5	1
US-10	Como estudante, quero buscar e filtrar vagas, para encontrar oportunidades.	Must	3	1
US-11	Como empresa, quero encerrar uma vaga, para parar de receber candidaturas.	Should	1	1
US-12	Como estudante, quero me candidatar a uma vaga, para concorrer ao estágio.	Must	3	2
US-13	Como empresa, quero analisar, aprovar ou rejeitar candidaturas pela máquina de estados, para conduzir a seleção.	Must	5	2
US-14	Como coordenação, quero que as regras da Lei 11.788/2008 sejam validadas automaticamente, para garantir a conformidade.	Must	5	2
US-15	Como estudante ou empresa, quero ver candidaturas em kanban por status, para acompanhar o processo.	Should	3	2

ID	História	MoSCoW	SP	Sprint
US-16	Como estudante e empresa, quero o TCE gerado automaticamente em PDF após a aprovação, para formalizar o estágio.	Must	8	2
US-17	Como coordenação, quero aprovar o TCE, para que o estágio passe a ativo.	Must	3	2
US-18	Como estudante, quero enviar relatórios de atividades com versões, para cumprir a periodicidade legal.	Must	5	2
US-19	Como usuário, quero notificações a cada transição, para ficar informado.	Must	5	2
US-20	Como estudante, quero vagas recomendadas com a explicação do porquê, para escolher onde me candidatar.	Must	8	3
US-21	Como empresa, quero o ranking de candidatos de uma vaga, para priorizar a análise.	Must	5	3
US-22	Como equipe, quero avaliar o recomendador offline com precision@5 e nDCG@5 contra *baselines*.	Should	3	3
US-23	Como estudante, quero um assistente de IA que explique a vaga, sugira melhorias e gere rascunho de carta.	Could	5	3
US-24	Como coordenação, quero um agente de acompanhamento que alerte sobre prazos.	Should	5	3

ID	História	MoSCoW	SP	Sprint
US-25	Como empresa, quero um agente de triagem que resuma o ranking de candidatos.	Could	3	3
US-26	Como pessoa avaliadora, quero uma tela com o status de todos os microsserviços.	Must	3	4
US-27	Como pessoa avaliadora, quero ver na interface quais microsserviços forneceram cada dado.	Must	3	4
US-28	Como equipe, quero um teste e2e do fluxo principal via gateway.	Must	5	4
US-29	Como equipe, quero o relatório gerado automaticamente em DOCX e PDF com prints.	Must	5	4
US-30	Como usuário, quero que o sistema continue funcionando se recomendação ou agentes caírem.	Should	3	4
US-31	Como coordenação, quero integração com o sistema acadêmico e assinatura digital.	Won’t	13	—

5.2 Sprint Backlogs

Tabela 13 — Sprint Backlogs (detalhes em `docs/scrum/sprint-N.md`).

Sprint	Histórias (SP)	Principais tarefas técnicas
0	US-01 (5), US-02 (8), US-03 (8), US-04 (3), US-05 (3) — **27 SP**	Componentes `eh_*`; gateway com rotas, políticas e `/services`; Compose; Makefile; imagem `tools`; CI; ADRs 001–006.
1	US-06 (5), US-07 (3), US-08 (2), US-09 (5), US-10 (3), US-11 (1) — **19 SP**	`user-service`; `internship-service`; migrações Alembic; frontend (login, dashboard, vagas, perfis); ADR-007.

Sprint	Histórias (SP)	Principais tarefas técnicas
2	US-12 (3), US-13 (5), US-14 (5), US-15 (3), US-16 (8), US-17 (3), US-18 (5), US-19 (5) — **37 SP**	Domínio puro (estados + lei); `application-service`; `document-service` (PDF); `notification-service`; saga; kanban, TCE, relatórios, notificações; ADR-008/009.
3	US-20 (8), US-21 (5), US-22 (3), US-23 (5), US-24 (5), US-25 (3) — **29 SP**	Recomendador + read model + *rebuild*; dados sintéticos e avaliação; `agent-service`; páginas de recomendação, ranking e assistente; ADR-010/011.
4	US-26 (3), US-27 (3), US-28 (5), US-29 (5), US-30 (3) — **19 SP**	Status, legenda e destaque; seed; e2e; prints Playwright; relatório (Pandoc/Mermaid); guia de estudo.

6 Projeto da solução

6.1 Arquitetura

O sistema tem oito microsserviços de negócio/infraestrutura de aplicação, um frontend, sete bancos de dados e um *broker* (Figura 2). O frontend só fala com o API Gateway; o gateway encaminha cada requisição ao serviço dono do prefixo. Os serviços se comunicam de forma síncrona (REST via `eh_http`) apenas para consultas pontuais e de forma assíncrona (eventos via `eh_events`) para tudo que muda o estado de outro contexto (Figura 3). A Figura 4 mostra a saga do fluxo principal e a Figura 5, a máquina de estados que ela percorre.

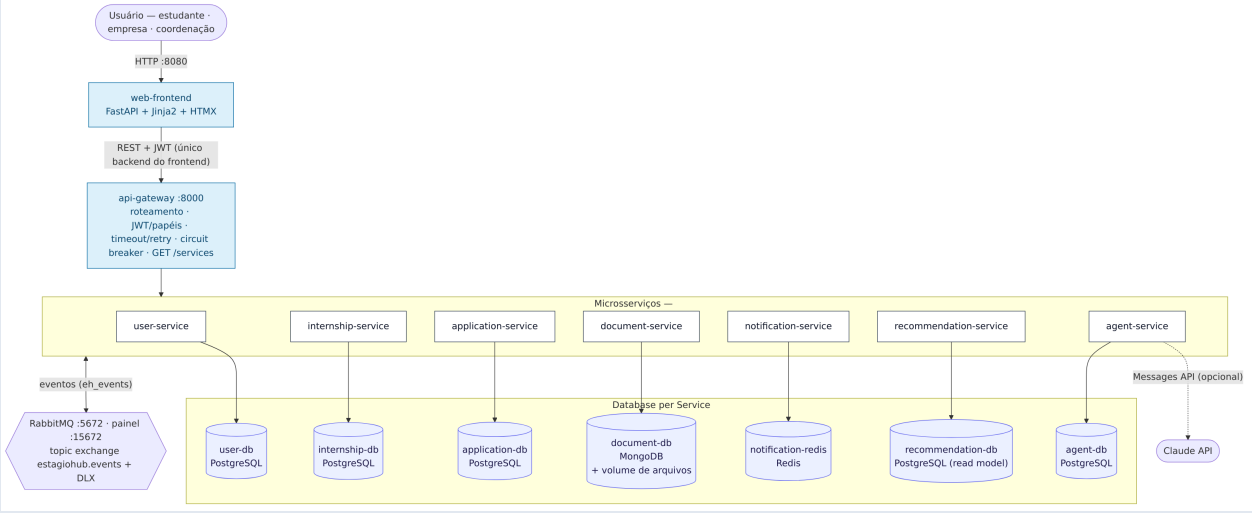


Figura 2 — Diagrama de contêineres (Docker Compose).

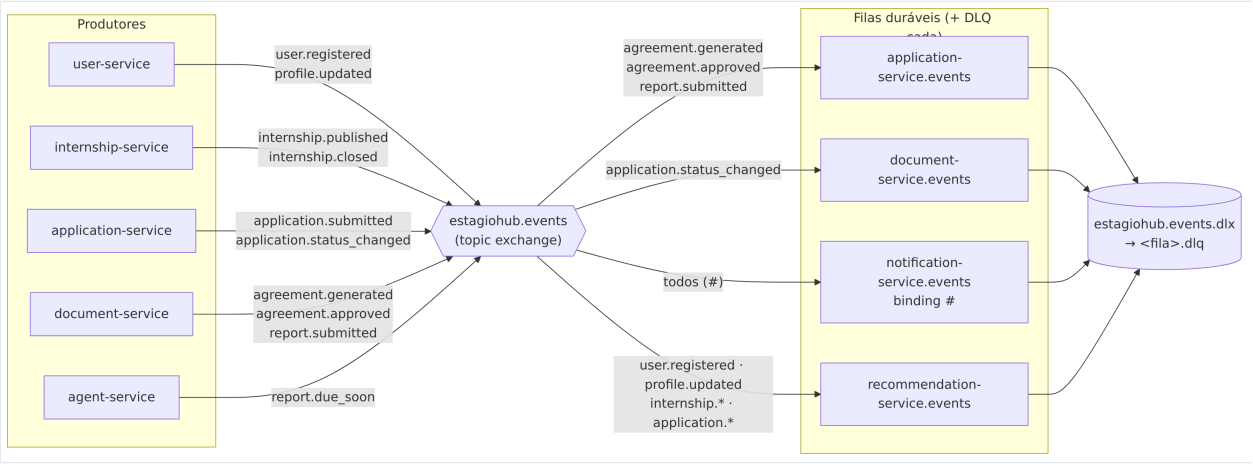


Figura 3 — Fluxo de eventos no RabbitMQ (topic exchange, filas e DLQ).

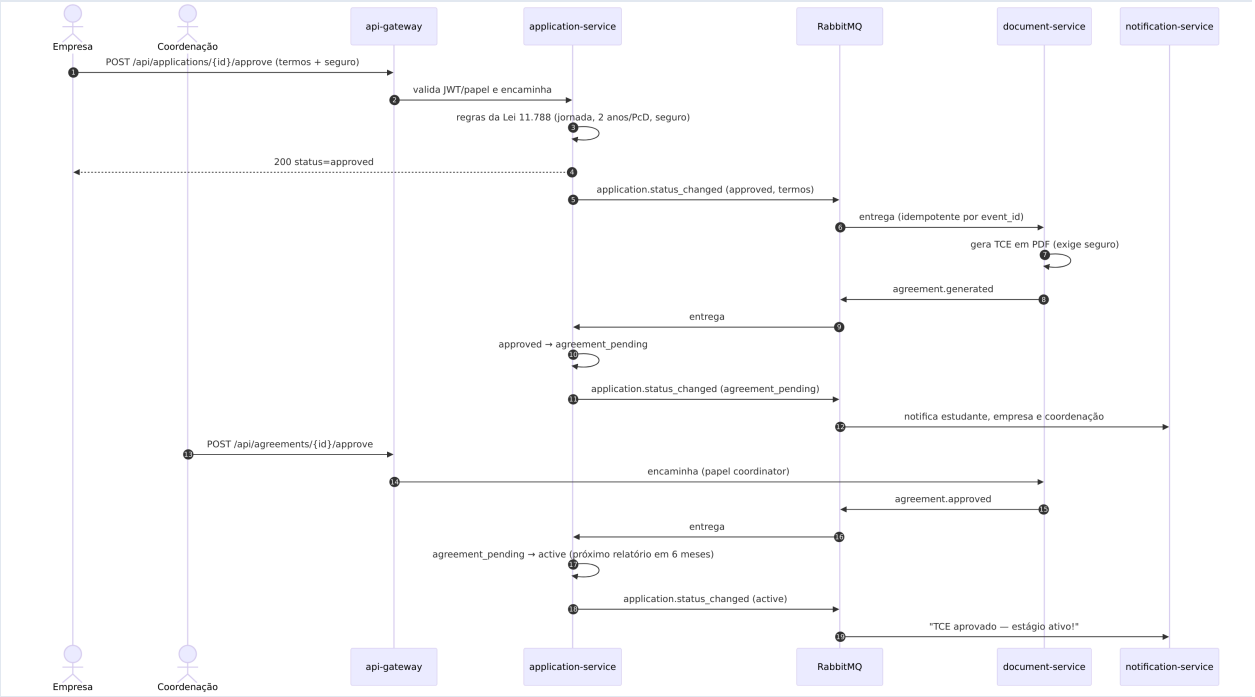


Figura 4 — Sequência da saga coreografada: aprovação → TCE → coordenação → estágio ativo.

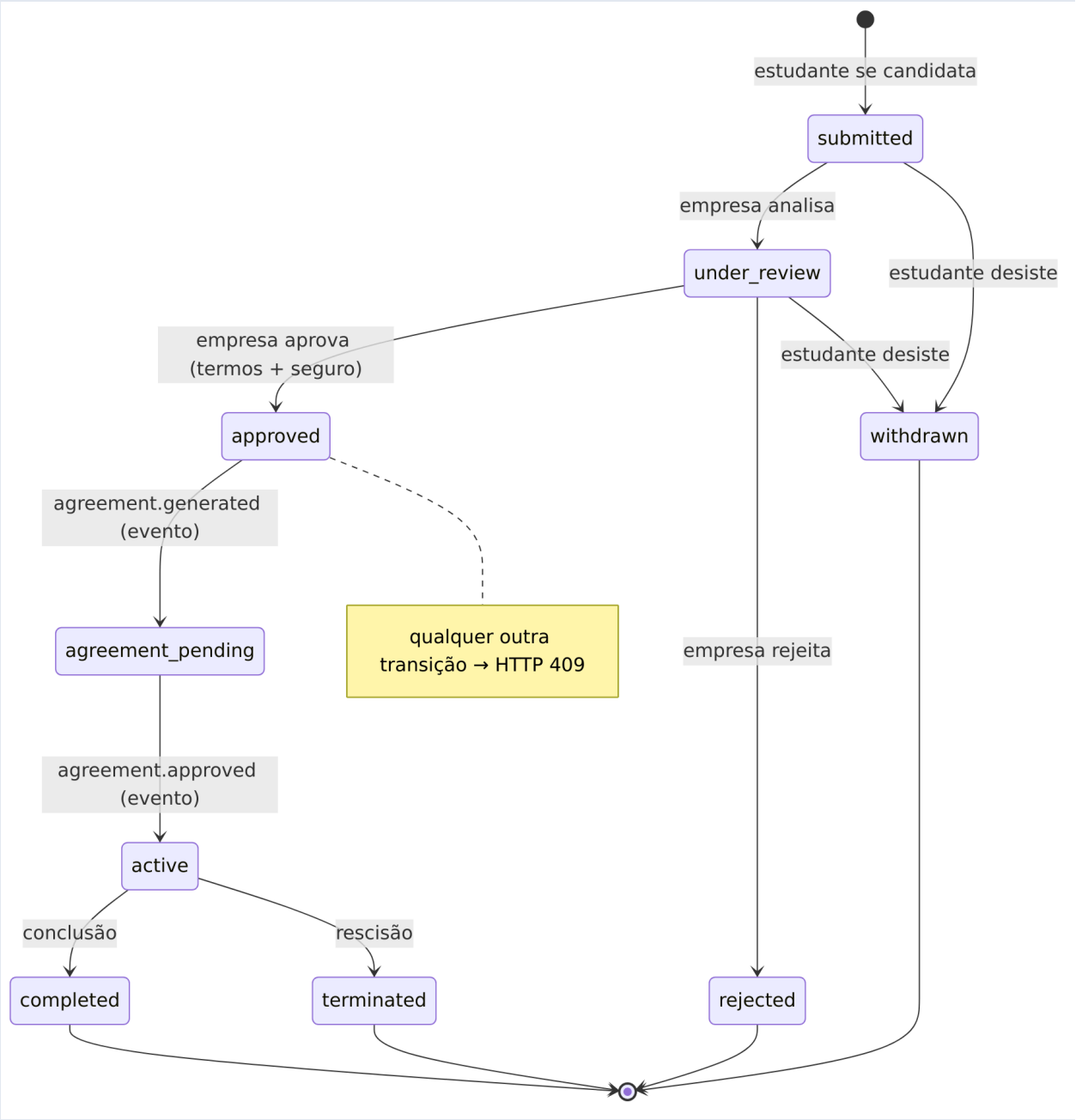


Figura 5 — Máquina de estados da candidatura e do estágio.

6.2 API Gateway

Responsabilidades: ponto único de entrada (porta 8000); roteamento por prefixo; validação do JWT e da política de papéis antes de a requisição chegar ao serviço; propagação do *correlation-id*; repasse do header `X-Served-By`; resiliência por upstream (*timeout*, *retry*, *circuit breaker*); conversão de falhas de upstream em 503 com o nome do serviço; agregação de saúde em `GET /services`; CORS restrito à origem do frontend.

Tabela 14 — Tabela de rotas do gateway (gerada de `api_gateway/routes.py`).

Prefixo público	Serviço de destino	Rota no serviço	Autenticação
`/api/auth/*`	user-service	`/auth/*`	pública

Prefixo público	Serviço de destino	Rota no serviço	Autenticação
`/api/users/*`	user-service	`/users/*`	JWT
`/api/profiles/*`	user-service	`/profiles/*`	JWT
`/api/internships/*`	internship-service	`/internships/*`	JWT
`/api/applications/*`	application-service	`/applications/*`	JWT
`/api/placements/*`	application-service	`/placements/*`	JWT
`/api/agreements/*`	document-service	`/agreements/*`	JWT
`/api/reports/*`	document-service	`/reports/*`	JWT
`/api/notifications/*`	notification-service	`/notifications/*`	JWT
`/api/recommendations/*`	recommendation-service	`/recommendations/*`	JWT
`/api/agents/*`	agent-service	`/agents/*`	JWT
`GET /services`	(gateway)	—	pública — agrega `/ready` e `/version`
`GET /routes`	(gateway)	—	pública — esta tabela

Tabela 15 — Política de papéis aplicada na borda (demais rotas: qualquer usuário autenticado; os serviços revalidam).

Método	Caminho (regex)	Papéis permitidos
POST	`/api/internships`	empresa
GET	`/api/internships/mine`	empresa
POST	`/api/internships/\d+/close`	empresa, coordenação
PUT	`/api/profiles/students/me`	estudante
PUT	`/api/profiles/companies/me`	empresa
POST	`/api/applications`	estudante
POST	`/api/applications/\d+/(review|approve|reject)`	empresa
POST	`/api/applications/\d+/withdraw`	estudante
POST	`/api/applications/\d+/(complete|terminate)`	empresa, coordenação
POST	`/api/agreements/[\w-]+/approve`	coordenação
POST	`/api/reports(/[\w-]+/versions)?`	estudante
POST	`/api/reports/[\w-]+/review`	coordenação
GET	`/api/recommendations/students/\d+/internships`	estudante, coordenação
GET	`/api/recommendations/internships/\d+/candidates`	empresa, coordenação
POST	`/api/recommendations/rebuild`	coordenação
*	`/api/agents/application-assistant(/.*)?`	estudante
*	`/api/agents/screening(/.*)?`	empresa
*	`/api/agents/follow-up(/.*)?`	coordenação

Autenticação e autorização — `services/api-gateway/api_gateway/proxy.py`:

```python
def authenticate(request: Request, auth: JWTAuth) -> Principal | JSONResponse:
    """Validate the bearer token; return the principal or a 401 response."""
    header = request.headers.get("authorization", "")
    scheme, _, token = header.partition(" ")
    if scheme.lower() != "bearer" or not token:
        return _error(401, "Autenticação necessária")
    try:
        return auth.decode(token)
    except TokenError:
        return _error(401, "Token inválido ou expirado")


def authorise(request: Request, principal: Principal) -> JSONResponse | None:
    """Apply the role policy table; return a 403 response when forbidden."""
    roles = allowed_roles(request.method, request.url.path)
    if roles is None or principal.role in roles or principal.role is Role.SERVICE:
        return None
    return _error(403, "Seu papel não tem permissão para esta operação", role=principal.role.value)
```

Roteamento — `services/api-gateway/api_gateway/routes.py` e `main.py`:

```python
def resolve_route(path: str) -> Route | None:
    """Return the route serving ``path`` (longest prefix wins) or ``None``."""
    candidates = [r for r in ROUTES if path == r.prefix or path.startswith(r.prefix + "/")]
    return max(candidates, key=lambda r: len(r.prefix), default=None)

@app.api_route("/api/{path:path}", methods=PROXY_METHODS, include_in_schema=False)
async def proxy(request: Request, path: str) -> Response:
    """Authenticate, authorise and forward any ``/api/*`` request."""
    route = resolve_route(request.url.path)
    if route is None:
        return JSONResponse({"detail": "Rota não encontrada"}, status_code=404)
    principal = None
    if not route.public:
        principal = authenticate(request, auth)
        if isinstance(principal, JSONResponse):
            return principal
        if (denied := authorise(request, principal)) is not None:
            return denied
    return await forward(request, route, clients[route.service], principal)
```

Agregação de saúde — `services/api-gateway/api_gateway/health_aggregator.py`:

```python
async def probe_service(client: httpx.AsyncClient, name: str, base_url: str) -> dict[str, Any]:
    """Probe one service and classify it as ``up``, ``degraded`` or ``down``.

    * ``up`` – ``/ready`` answered 200 (database and broker reachable);
    * ``degraded`` – the process answers but a dependency is not ready;
    * ``down`` – the service could not be reached at all.
    """
    ...
    ready = await client.get(f"{base_url}/ready")
    result["latency_ms"] = round((time.perf_counter() - started) * 1000, 1)
    body = ready.json()
    result.update(status="up" if ready.status_code == 200 else "degraded",
                  database=body.get("database"), uptime_seconds=body.get("uptime_seconds"),
                  checks=body.get("checks", {}))
```

Arquivos: `services/api-gateway/api_gateway/`.

6.3 Serviços e componentes

Todos os serviços expõem OpenAPI em `/docs`, usam migrações Alembic (os com PostgreSQL), Dockerfile *multi-stage* com imagem *slim* e usuário não-root, e respondem com `X-Served-By: <serviço>`. As tabelas de endpoints abaixo são geradas a partir dos *routers* FastAPI, inclusive o papel exigido (lido da dependência `require_role`).

6.3.1 user-service

Responsabilidade: contas, papéis, emissão de JWT e perfis (estudante, empresa). Cadastro de coordenação exige token de coordenação ou de serviço. **Banco:** PostgreSQL `users` — tabelas `users`, `student_profiles`, `company_profiles`. **Eventos publicados:** `user.registered`, `profile.updated` (perfil completo, para o modelo de leitura). **Consumidos:** nenhum. **Componentes:** `eh_auth`, `eh_db`, `eh_events`, `eh_observability`.

Tabela 16 — Endpoints do user-service.

Método	Rota	Papel exigido	Descrição
POST	`/auth/register`	público (token opcional)	Cadastrar conta (estudante ou empresa; coordenação exige token de coordenação)
POST	`/auth/login`	público	Login
GET	`/users/me`	autenticado (qualquer papel)	Conta autenticada
GET	`/users`	coordenação, serviço interno	Listar contas (coordenação/serviços)
GET	`/users/{user_id}`	autenticado (qualquer papel)	Conta por id
GET	`/profiles/students/me`	estudante	Meu perfil acadêmico
PUT	`/profiles/students/me`	estudante	Atualizar meu perfil acadêmico (publica profile.updated)
GET	`/profiles/students`	empresa, coordenação, serviço interno	Perfis de vários estudantes (empresa, coordenação, serviços)
GET	`/profiles/students/{user_id}`	autenticado (qualquer papel)	Perfil de um estudante
GET	`/profiles/companies/me`	empresa	Perfil da minha empresa
PUT	`/profiles/companies/me`	empresa	Atualizar perfil da empresa
GET	`/profiles/companies/{user_id}`	autenticado (qualquer papel)	Perfil público de uma empresa
GET	`/health`, `/ready`, `/version`	público	Health Check API (`eh_observability`)

```python
@router.post("/auth/login", response_model=TokenResponse, tags=["auth"], summary="Login")
async def login(body: LoginRequest, session: AsyncSession = Depends(get_session),
                auth: JWTAuth = Depends(get_auth)) -> TokenResponse:
    """Check the password (argon2) and issue a JWT carrying the user's role."""
    user = await UserRepository(session).by_email(body.email)
    if user is None or not user.is_active or not verify_password(body.password, user.password_hash):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "E-mail ou senha inválidos")
    token = auth.create_token(user_id=user.id, role=user.role, name=user.full_name, email=user.email)
    return TokenResponse(access_token=token, expires_in=auth.expires_minutes * 60,
                         user=UserOut.model_validate(user))
```

Arquivos: `services/user-service/user_service/`.

6.3.2 internship-service

Responsabilidade: publicação, busca (texto, curso, modalidade, cidade, paginação), estatísticas e encerramento de vagas, validando a jornada legal na API (pydantic) e no banco (*check constraints*). **Banco:** PostgreSQL `internships` — tabela `internships`. **Eventos publicados:** `internship.published` (vaga completa), `internship.closed`. **Consumidos:** nenhum. **Componentes:** `eh_auth`, `eh_db`, `eh_events`, `eh_observability`.

Tabela 17 — Endpoints do internship-service.

Método	Rota	Papel exigido	Descrição
GET	`/internships`	autenticado (qualquer papel)	Buscar vagas
GET	`/internships/mine`	empresa	Vagas da empresa
GET	`/internships/stats`	autenticado (qualquer papel)	Indicadores
GET	`/internships/{internship_id}`	autenticado (qualquer papel)	Detalhe
POST	`/internships`	empresa	Publicar vaga (empresa; publica internship.published)
POST	`/internships/{internship_id}/close`	empresa, coordenação	Encerrar vaga (empresa dona ou coordenação; publica internship.closed)
GET	`/health`, `/ready`, `/version`	público	Health Check API (`eh_observability`)

```python
# Lei nº 11.788/2008, art. 10, II — higher education: up to 6 h/day and 30 h/week.
class InternshipIn(BaseModel):
    """Data required to publish an internship offer."""
    hours_per_day: int = Field(ge=1, le=MAX_HOURS_PER_DAY)
    hours_per_week: int = Field(ge=1, le=MAX_HOURS_PER_WEEK)
    ...
    @model_validator(mode="after")
    def check_workload(self) -> "InternshipIn":
        """Weekly hours cannot exceed what the daily limit allows in a 6-day week."""
        if self.hours_per_week > self.hours_per_day * 6:
            raise ValueError("carga semanal incompatível com a carga diária")
        return self
```

Arquivos: `services/internship-service/internship_service/`.

6.3.3 application-service

Responsabilidade: candidaturas e ciclo de vida do estágio: máquina de estados (Figura 5; transição inválida → 409), regras legais (jornada, 2 anos cumulativos na mesma concedente exceto PcD, seguro, relatório semestral, elegibilidade por curso/período/disponibilidade), histórico de transições e lista de estágios com prazos. Consulta `internship-service` e `user-service` via REST resiliente ao receber uma candidatura. **Banco:** PostgreSQL `applications` — `applications`, `placements`, `status_changes`, `processed_events`. **Eventos publicados:** `application.submitted`, `application.status_changed`. **Consumidos:** `agreement.generated`, `agreement.approved`, `report.submitted`. **Componentes:** `eh_auth`, `eh_db`, `eh_events`, `eh_http`, `eh_observability`.

Tabela 18 — Endpoints do application-service.

Método	Rota	Papel exigido	Descrição
POST	`/applications`	estudante	Candidatar-se a uma vaga (estudante; publica application.submitted)
GET	`/applications`	autenticado (qualquer papel)	Listar candidaturas visíveis ao papel (filtros: vaga, status)
GET	`/applications/board`	autenticado (qualquer papel)	Candidaturas agrupadas por status (kanban)
GET	`/applications/stats`	autenticado (qualquer papel)	Contagem por status (escopo do papel)
GET	`/applications/{application_id}`	autenticado (qualquer papel)	Detalhe da candidatura (com termos e histórico)
POST	`/applications/{application_id}/review`	empresa	Iniciar análise (empresa): submitted → under_review
POST	`/applications/{application_id}/approve`	empresa	Aprovar candidato com os termos do estágio (empresa): under_review → approved
POST	`/applications/{application_id}/reject`	empresa	Rejeitar (empresa): under_review → rejected
POST	`/applications/{application_id}/withdraw`	estudante	Desistir (estudante): submitted/under_review → withdrawn
POST	`/applications/{application_id}/complete`	empresa, coordenação	Concluir estágio (empresa/coordenação): active → completed
POST	`/applications/{application_id}/terminate`	empresa, coordenação	Rescindir estágio (empresa/coordenação): active → terminated
GET	`/placements`	autenticado (qualquer papel)	Estágios (com prazos de relatório) visíveis ao papel

Método	Rota	Papel exigido	Descrição
GET	`/health`, `/ready`, `/version`	público	Health Check API (`eh_observability`)

```python
TRANSITIONS: dict[Status, frozenset[Status]] = {
    Status.SUBMITTED: frozenset({Status.UNDER_REVIEW, Status.WITHDRAWN}),
    Status.UNDER_REVIEW: frozenset({Status.APPROVED, Status.REJECTED, Status.WITHDRAWN}),
    Status.APPROVED: frozenset({Status.AGREEMENT_PENDING}),
    Status.AGREEMENT_PENDING: frozenset({Status.ACTIVE}),
    Status.ACTIVE: frozenset({Status.COMPLETED, Status.TERMINATED}),
    ...  # rejected, withdrawn, completed, terminated: final states
}

def validate_duration(start: date, end: date, *, is_pcd: bool, previous_days: int = 0) -> None:
    """Art. 11: at most 2 years at the same company (cumulative), except for PcD interns."""
    if end <= start:
        raise LegalRuleViolationError("dates_invalid", "A data de término deve ser posterior à de início.")
    if is_pcd:
        return
    if end > max_end_date(start, previous_days):
        raise LegalRuleViolationError("duration_exceeded", "A duração ultrapassa 2 anos ...")
```

Arquivos: `services/application-service/application_service/` (`domain.py` concentra as regras em funções puras, testadas exaustivamente).

6.3.4 document-service

Responsabilidade: geração do TCE em PDF (ReportLab) ao receber a aprovação, pré-visualização PNG (pypdfium2), aprovação pela coordenação e relatórios de atividades com versões (PDF até 5 MB). O TCE só é emitido com seguro contra acidentes pessoais. **Banco:** MongoDB `documents` (coleções `agreements`, `reports`, `processed_events`) + volume `document-files` para os arquivos. **Eventos publicados:** `agreement.generated`, `agreement.approved`, `report.submitted`. **Consumidos:** `application.status_changed`. **Componentes:** `eh_auth`, `eh_events`, `eh_http`, `eh_observability`.

Tabela 19 — Endpoints do document-service.

Método	Rota	Papel exigido	Descrição
GET	`/agreements`	autenticado (qualquer papel)	Listar TCEs visíveis ao papel
GET	`/agreements/{agreement_id}`	autenticado (qualquer papel)	Metadados do TCE
GET	`/agreements/{agreement_id}/pdf`	autenticado (qualquer papel)	PDF do TCE
GET	`/agreements/{agreement_id}/preview.png`	autenticado (qualquer papel)	Pré-visualização (PNG da 1ª página)
POST	`/agreements/{agreement_id}/approve`	coordenação	Aprovar TCE (coordenação; publica agreement.approved)

Método	Rota	Papel exigido	Descrição
POST	`/reports`	estudante	Enviar relatório de atividades (estudante; publica report.submitted)
POST	`/reports/{report_id}/versions`	estudante	Enviar nova versão do relatório (versões anteriores são preservadas)
GET	`/reports`	autenticado (qualquer papel)	Listar relatórios visíveis ao papel
GET	`/reports/{report_id}`	autenticado (qualquer papel)	Detalhe do relatório
GET	`/reports/{report_id}/versions/{version}/file`	autenticado (qualquer papel)	Baixar uma versão do relatório
POST	`/reports/{report_id}/review`	coordenação	Registrar revisão da coordenação
GET	`/health`, `/ready`, `/version`	público	Health Check API (`eh_observability`)

```
class AgreementGenerator:
    """Saga step: ``application.status_changed(approved)`` → TCE PDF → ``agreement.generated``."""

    async def __call__(self, event: Event) -> None:
        """Event handler registered in the consumer."""
        if event.event_type != EventType.APPLICATION_STATUS_CHANGED:
            return
        payload: ApplicationStatusChanged = typed_payload(event)
        if payload.to_status == "approved" and payload.placement is not None:
            await self.generate(payload)

def render_agreement_pdf(data: AgreementData) -> bytes:
    """Render the TCE as an A4 PDF.

    Raises:
        MissingInsuranceError: insurer or policy number missing (the TCE is not issued).
    """
```

Arquivos: `services/document-service/document_service/`.

6.3.5 notification-service

Responsabilidade: consumir **todos** os eventos (*binding* `#`) e manter a caixa de notificações de cada usuário (e a caixa compartilhada da coordenação, com leitura individual); receber notificações diretas dos agentes. **Banco:** Redis (listas `inbox:*`, conjuntos `read:*`, marcadores `processed:*` com TTL). **Eventos publicados:** nenhum. **Consumidos:** todos. **Componentes:** `eh_auth`, `eh_events`, `eh_observability`.

Tabela 20 — Endpoints do notification-service.

Método	Rota	Papel exigido	Descrição
GET	`/notifications`	autenticado (qualquer papel)	Minhas notificações (pessoais + do meu papel)

Método	Rota	Papel exigido	Descrição
GET	`/notifications/unread-count`	autenticado (qualquer papel)	Quantidade de não lidas
POST	`/notifications/read`	autenticado (qualquer papel)	Marcar como lidas
POST	`/notifications/read-all`	autenticado (qualquer papel)	Marcar todas como lidas
POST	`/notifications`	coordenação, serviço interno	Enviar notificação direta (serviços internos e coordenação)
GET	`/health`, `/ready`, `/version`	público	Health Check API (`eh_observability`)

```python
def notices_for(event: Event) -> list[Notice]:
    """Translate a domain event into the notifications of the actors involved."""
    p = typed_payload(event)
    match event.event_type:
        case EventType.AGREEMENT_GENERATED:
            link = f"/termos/{p.agreement_id}"
            return [
                Notice(user(p.student_id), "TCE gerado", "Seu Termo de Compromisso foi gerado ...", "info",
          link),
                Notice(user(p.company_id), "TCE gerado", f"Termo de {p.student_name} gerado ...", "info",
          link),
                Notice(COORDINATION, "TCE aguardando aprovação", ..., "warning", link),
            ]
        ...
```

Arquivos: `services/notification-service/notification_service/`.

6.3.6 recommendation-service

Responsabilidade: recomendação híbrida e explicável (seção 6.4) e ranking de candidatos, sobre um modelo de leitura próprio; reconstrução do modelo via REST. **Banco:** PostgreSQL `recommendations` — `rm_students`, `rm_internships`, `rm_applications`, `processed_events`. **Eventos publicados:** nenhum. **Consumidos:** `user.registered`, `profile.updated`, `internship.published`, `internship.closed`, `application.submitted`, `application.status_changed`. **Componentes:** `eh_auth`, `eh_db`, `eh_events`, `eh_http`, `eh_observability`.

Tabela 21 — Endpoints do recommendation-service.

Método	Rota	Papel exigido	Descrição
GET	`/recommendations/students/{student_id}/internships`	estudante, coordenação, serviço interno	Vagas recomendadas para um estudante (com explicação)
GET	`/recommendations/internships/{internship_id}/candidates`	empresa, coordenação, serviço interno	Ranking de candidatos de uma vaga (empresa dona, coordenação)
GET	`/recommendations/stats`	autenticado (qualquer papel)	Tamanho do modelo de leitura e pesos

Método	Rota	Papel exigido	Descrição
POST	`/recommendations/rebuild`	coordenação, serviço interno	Reconstruir o modelo de leitura a partir dos serviços de origem
GET	`/health`, `/ready`, `/version`	público	Health Check API (`eh_observability`)

Arquivos: `services/recommendation-service/recommendation_service/` (código na seção 6.4).

6.3.7 agent-service

Responsabilidade: três agentes de IA — Assistente de Candidatura (estudante), Agente de Acompanhamento (coordenação; agendado com APScheduler e sob demanda) e Agente de Triagem (empresa) — com Claude via SDK oficial da Anthropic (Messages API com *tool use*), no máximo 6 passos por execução, *timeout* e fallback determinístico em modo regras (selo “IA em modo regras” na interface). *Guardrails*: as ferramentas só leem, sugerem e notificam; nenhuma altera status de candidatura. Toda chamada de ferramenta é gravada e exibida como trilha de execução. **Banco:** PostgreSQL `agents` — `agent_runs`, `agent_steps`, `alerts_sent`. **Eventos publicados:** `report.due_soon`. **Consumidos:** nenhum. **Componentes:** `eh_auth`, `eh_db`, `eh_events`, `eh_http`, `eh_observability`.

Tabela 22 — Endpoints do agent-service.

Método	Rota	Papel exigido	Descrição
GET	`/agents/status`	autenticado (qualquer papel)	Modo dos agentes (IA ou regras), modelo e agendamento
POST	`/agents/application-assistant/run`	estudante	Assistente de Candidatura (estudante)
POST	`/agents/screening/run`	empresa	Agente de Triagem (empresa)
POST	`/agents/follow-up/run`	coordenação	Agente de Acompanhamento sob demanda (coordenação)
GET	`/agents/runs`	autenticado (qualquer papel)	Histórico de execuções
GET	`/agents/runs/{run_id}`	autenticado (qualquer papel)	Execução com a trilha de ferramentas
GET	`/health`, `/ready`, `/version`	público	Health Check API (`eh_observability`)

```python
async def run_llm(spec, trail, prompt, llm, model, max_steps) -> AgentOutput:
    """Manual tool-use loop limited to ``max_steps`` model calls.

    Every model call and tool call is recorded in ``trail``. Thinking/tool_use
    blocks are sent back unchanged (``response.content``).
    """
    tools = [TOOLS[name].to_anthropic() for name in spec.tools]
    messages = [{"role": "user", "content": prompt}]
    for _ in range(max_steps):
        response = await llm.create(model=model, max_tokens=8000, system=spec.system_prompt,
                                    tools=tools, output_config={"effort": "medium"}, messages=messages)
        if response.stop_reason == "refusal":
            raise LlmUnavailableError("o modelo recusou a solicitação")
        tool_uses = [b for b in response.content if b.type == "tool_use"]
        if response.stop_reason != "tool_use" or not tool_uses:
            ...  # final answer (+ cover letter between <carta> tags)
        messages.append({"role": "assistant", "content": response.content})
        results = [...]  # trail.call(...) for each tool_use → tool_result (is_error on failure)
        messages.append({"role": "user", "content": results})
    raise StepLimitExceededError(f"limite de {max_steps} passos atingido")
```

Arquivos: `services/agent-service/agent_service/`.

6.3.8 web-frontend

Responsabilidade: interface web (porta 8080) em FastAPI + Jinja2 + HTMX (assets locais, sem CDN). Consome **apenas** o gateway; guarda o JWT em *cookie* HttpOnly; cada painel exibe o serviço que forneceu os dados (header `X-Served-By`) e o *toggle* “Mostrar microsserviços” destaca cada painel com a cor do serviço; painéis independentes degradam isoladamente. **Banco:** nenhum. **Componentes:** `eh_http`, `eh_observability`, `eh_ui`. Páginas: login/cadastro, dashboard por papel (com legenda de serviços), vagas, recomendações, candidaturas (kanban), aprovação com termos, ranking de candidatos, TCE (pré-visualização e aprovação), relatórios, notificações, assistente de IA (trilha) e status dos microsserviços.

```python
async def request(self, method: str, path: str, *, token: str | None = None, **kwargs) -> ApiResult:
    """Send a request and decode the JSON body into an :class:`ApiResult`."""
    response = await self.raw(method, path, token=token, **kwargs)
    if response is None:
        return ApiResult(503, {"service": "api-gateway"}, error="API Gateway indisponível")
    ...
    result = ApiResult(response.status_code, data, response.headers.get("x-served-by"))
```

Arquivos: `frontend/web/web_frontend/`.

6.3.9 Componentes

Tabela 23 — Interfaces fornecidas e requeridas dos componentes (detalhes e exemplos nos READMEs de `components/`).

Componente	Interface fornecida	Interface requerida	Reutilizado por
`eh_auth`	`JWTAuth`, `Principal`, `Role`, `hash_password`/`verify_password`, `install_auth`, `get_current_principal`, `require_role`	PyJWT, argon2-cffi, FastAPI; segredo ≥ 32 caracteres	gateway e todos os serviços de negócio

Componente	Interface fornecida	Interface requerida	Reutilizado por
`eh_events`	`Event`, `EventType`, contratos de *payload*, `make_event`/`parse_event`, `EventPublisher`, `InMemoryPublisher`, `EventConsumer`, `IdempotencyStore`	pydantic, aio-pika, broker AMQP, uma implementação de `IdempotencyStore`	7 serviços
`eh_db`	`Base`, `TimestampMixin`, `Database`, `BaseRepository`, `ProcessedEvent`, `SqlIdempotencyStore`, `run_migrations`	SQLAlchemy 2 async, Alembic, driver (asyncpg/aiosqlite)	5 serviços com PostgreSQL
`eh_observability`	`configure_logging`, `JsonFormatter`, `ObservabilityMiddleware`, `get_correlation_id`, `create_health_router`, `setup_observability`	FastAPI/Starlette	todos (inclusive frontend)
`eh_http`	`ResilientClient`, `CircuitBreaker`, `CircuitOpenError`, `UpstreamUnavailableError`	httpx; opcionalmente um `header_provider`	gateway, application, document, recommendation, agent, frontend
`eh_ui`	macros `card`, `table`, `service_badge`, `alert`, `status_pill`, `score_bar`, `stat`, `service_card`, `empty_state`; filtros pt-BR; `eh_ui.css`	Jinja2	web-frontend

Trechos principais:

```python
# eh_auth — components/eh_auth/src/eh_auth/dependencies.py
def require_role(*roles: Role | str) -> Callable[..., Awaitable[Principal]]:
    """Build a dependency that only lets callers with one of ``roles`` through.

    Responds ``401`` without a valid token and ``403`` for other roles.
    """
    allowed = frozenset(Role(r) for r in roles)

    async def dependency(principal: Principal = Depends(get_current_principal)) -> Principal:
        if principal.role not in allowed:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "Seu papel não tem permissão para esta operação")
        return principal

    return dependency
```

```python
# eh_events — components/eh_events/src/eh_events/consumer.py (EventConsumer.handle_message)
try:
    event = parse_event(message.body)
except (ValidationError, ValueError, KeyError):
    await message.reject(requeue=False)          # poison message → DLQ
    return "dead_lettered"
if await self.idempotency.seen(event.event_id):  # duplicate delivery
    await message.ack()
    return "duplicate"
try:
    await self.handler(event)
except Exception:
    retries = int((message.headers or {}).get(RETRY_HEADER, 0))
    if retries < self.max_retries:               # retry with exponential backoff
        await asyncio.sleep(self.retry_delay * 2**retries)
        await self._republish(message, retries + 1)
        await message.ack()
        return "retried"
    await message.reject(requeue=False)          # exhausted → DLQ
    return "dead_lettered"
await self.idempotency.mark(event.event_id, event.event_type.value)
await message.ack()
return "processed"
```

```python
# eh_http — components/eh_http/src/eh_http/client.py (ResilientClient.request, trecho)
if not self.breaker.allow_request():
    raise CircuitOpenError(self.name)            # fail fast while the circuit is open
for attempt in range(self.retries + 1):
    if attempt:
        await asyncio.sleep(self.backoff * 2 ** (attempt - 1))
    try:
        response = await self._client.request(method, url, headers=headers, **kwargs)
    except (httpx.ConnectError, httpx.ConnectTimeout) as exc:
        last_error = exc                         # nothing reached the upstream: safe to retry
        continue
    ...
```

```python
# eh_db — components/eh_db/src/eh_db/database.py
@asynccontextmanager
async def session(self) -> AsyncIterator[AsyncSession]:
    """Yield a session that commits on success and rolls back on error."""
    async with self.session_factory() as session:
        try:
            yield session
            await session.commit()
        except BaseException:
            await session.rollback()
            raise
```

```python
# eh_observability — components/eh_observability/src/eh_observability/setup.py
def setup_observability(app, service_name, version, *, checks=None, database=None) -> None:
    """Install :class:`ObservabilityMiddleware` and the health router on ``app``."""
    app.add_middleware(ObservabilityMiddleware, service_name=service_name)
    app.include_router(create_health_router(service_name, version, checks=checks, database=database))
```

```
{# eh_ui — components/eh_ui/src/eh_ui/templates/eh_ui/macros.html #}
{% macro card(title, service=None, subtitle=None, id=None, classes="", actions=None) -%}
<section class="eh-card {{ classes }}"{% if service %} data-service="{{ service }}"
         style="--svc: {{ service|service_color }}"{% endif %}>
  <header class="eh-card__header">
    <div><h2 class="eh-card__title">{{ title }}</h2> ... </div>
    <div class="eh-card__meta">{{ actions }}{{ service_badge(service) }}</div>
  </header>
  <div class="eh-card__body">{{ caller() if caller else "" }}</div>
</section>
{%- endmacro %}
```

Arquivos: `components/`.

6.4 Recomendação

Arquitetura. O `recommendation-service` mantém um **modelo de leitura** (CQRS leve) alimentado pelos eventos de usuários, vagas e candidaturas; nunca consulta o banco de outro serviço. O algoritmo é um módulo puro (`recommender.py`), sem I/O, reutilizado pelos *endpoints* e pela avaliação offline (Figura 6).

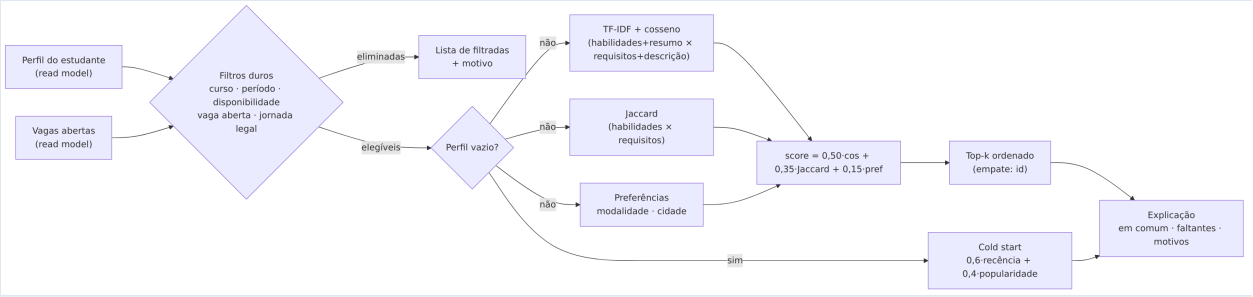


Figura 6 — Pipeline do recomendador híbrido.

Algoritmo.

1. **Filtros duros:** curso compatível (comparação sem acentos), período ≥ mínimo, carga semanal ≤ disponibilidade, vaga aberta e jornada legal. Vagas eliminadas voltam com o motivo.

2. **Score de conteúdo:** TF-IDF (unigramas e bigramas, *stop words* pt-BR, acentos removidos, *sublinear tf*) de *habilidades (peso 2) + resumo + curso* contra *requisitos (peso 2) + título + descrição*; similaridade do cosseno (MANNING; RAGHAVAN; SCHÜTZE, 2008), com scikit-learn (PEDREGOSA et al., 2011).

3. **Score de atributos:** índice de Jaccard entre habilidades exigidas e declaradas (normalizadas).

4. **Preferências:** +0,6 se a modalidade é a preferida (+0,3 se uma das partes é híbrida); +0,4 se a vaga é remota ou na cidade do estudante.

5. **Score final:**

$$score = w_1 \cdot \cos + w_2 \cdot Jaccard + w_3 \cdot pref, \quad w_1 = 0,50, w_2 = 0,35, w_3 = 0,15$$

Os pesos são configuráveis por `REC_WEIGHT_CONTENT`, `REC_WEIGHT_ATTRIBUTES` e `REC_WEIGHT_PREFERENCE` e normalizados para somar 1.

6. **Determinismo:** empates desfeitos pelo identificador; o mesmo estado gera sempre o mesmo resultado.

Explicabilidade. Cada item traz `matched_skills`, `missing_skills`, os componentes do *score* e um texto: “Você tem 3 de 4 habilidades exigidas (Python, SQL, Docker). Faltam: Git. Modalidade compatível com sua preferência. Seu perfil é 31% similar ao texto da vaga (TF-IDF).” Vagas eliminadas trazem o filtro responsável. O ranking de candidatos usa o mesmo cálculo no sentido inverso e mantém os candidatos inelegíveis no fim, com o motivo.

Cold start. Sem habilidades e sem resumo, o *score* passa a ser 0,6 · recência (meia-vida de 14 dias) + 0,4 · popularidade (candidaturas recebidas), com a explicação correspondente.

```python
# services/recommendation-service/recommendation_service/recommender.py (recommend_for_student, trecho)
w = weights.normalized()
cosines = content_scores(student_document(student), [offer_document(o) for o in eligible])
for offer, cos in zip(eligible, cosines, strict=True):
    matched = [r for r in offer.requirements if normalize(r) in student_skills]
    missing = [r for r in offer.requirements if normalize(r) not in student_skills]
    jac = jaccard(student.skills, offer.requirements)
    pref, pref_reasons = preference_score(student, offer)
    score = w.content * cos + w.attributes * jac + w.preference * pref
    ...
ranked.sort(key=lambda r: (-r.score, r.item_id))   # deterministic tie-break
```

Avaliação offline. `scripts/evaluate_recommender.py` (`make eval`) usa um conjunto **sintético e fictício** gerado por `scripts/synthetic_data.py` — o mesmo carregado pelo seed — com 50 estudantes, 30 vagas, 8 empresas e 10 “trilhas” de interesse (backend, dados, DevOps, frontend, mobile, embarcados, finanças, marketing, obras, UX). Os rótulos vêm da **trilha latente**, e não das habilidades vistas pelo recomendador: relevância 2 = mesma trilha e elegível; 1 = trilha adjacente e elegível; 0 = demais. As habilidades declaradas são ruidosas (parte vem de outra trilha do mesmo curso) e alguns resumos são genéricos. Métricas: precision@5 e nDCG@5 com ganhos 2^rel − 1 (JÄRVELIN; KEKÄLÄINEN, 2002).

Tabela 24 — Avaliação offline (dados sintéticos declarados).

Estratégia	Precision@5	nDCG@5
Baseline aleatório	0,151	0,151
Baseline só filtros (mais recentes)	0,708	0,825
Híbrido (0,50·cos + 0,35·Jaccard + 0,15·pref)	0,725	0,965
Ablação: só conteúdo (TF-IDF/cosseno)	0,721	0,978
Ablação: só atributos (Jaccard)	0,725	0,966

Dados **sintéticos** (seed 2026): 48 estudantes avaliados, 30 vagas, k = 5; baseline aleatório = média de 30 sorteios por estudante. Rótulos: 2 = mesma trilha latente e elegível; 1 = trilha adjacente e elegível; 0 = demais.

Discussão. O híbrido supera o *baseline* aleatório por larga margem e o só-filtros na **ordenação** (nDCG@5). A precision@5 dos dois fica próxima porque cada estudante tem poucas vagas elegíveis do seu curso, então ambos acabam recomendando quase o mesmo conjunto; a diferença está em colocar a vaga da trilha certa no topo. A ablação "só conteúdo" ficou ligeiramente acima do híbrido no nDCG@5 neste conjunto; os pesos **não** foram ajustados ao conjunto sintético para evitar sobreajuste, e o componente de atributos/preferências mantém a explicação "x de y habilidades" que o usuário entende. Limitação: os dados são sintéticos e favorecem métodos baseados em conteúdo.

Integração com os agentes. O Assistente de Candidatura usa a ferramenta `get_recommendations` (explicação, faltantes e filtros) para justificar a vaga, sugerir melhorias de perfil e redigir a carta; o Agente de Triagem usa `get_candidate_ranking` para resumir o ranking. Assim os agentes não inventam critérios: explicam a saída determinística do recomendador.

6.5 Serviços em execução

A Figura 7 mostra a tela "Status dos Microsserviços", alimentada por `GET /services` do gateway e atualizada a cada 5 segundos, com nome, status, versão, latência, banco, *uptime* e as verificações de banco e *broker* de cada serviço.

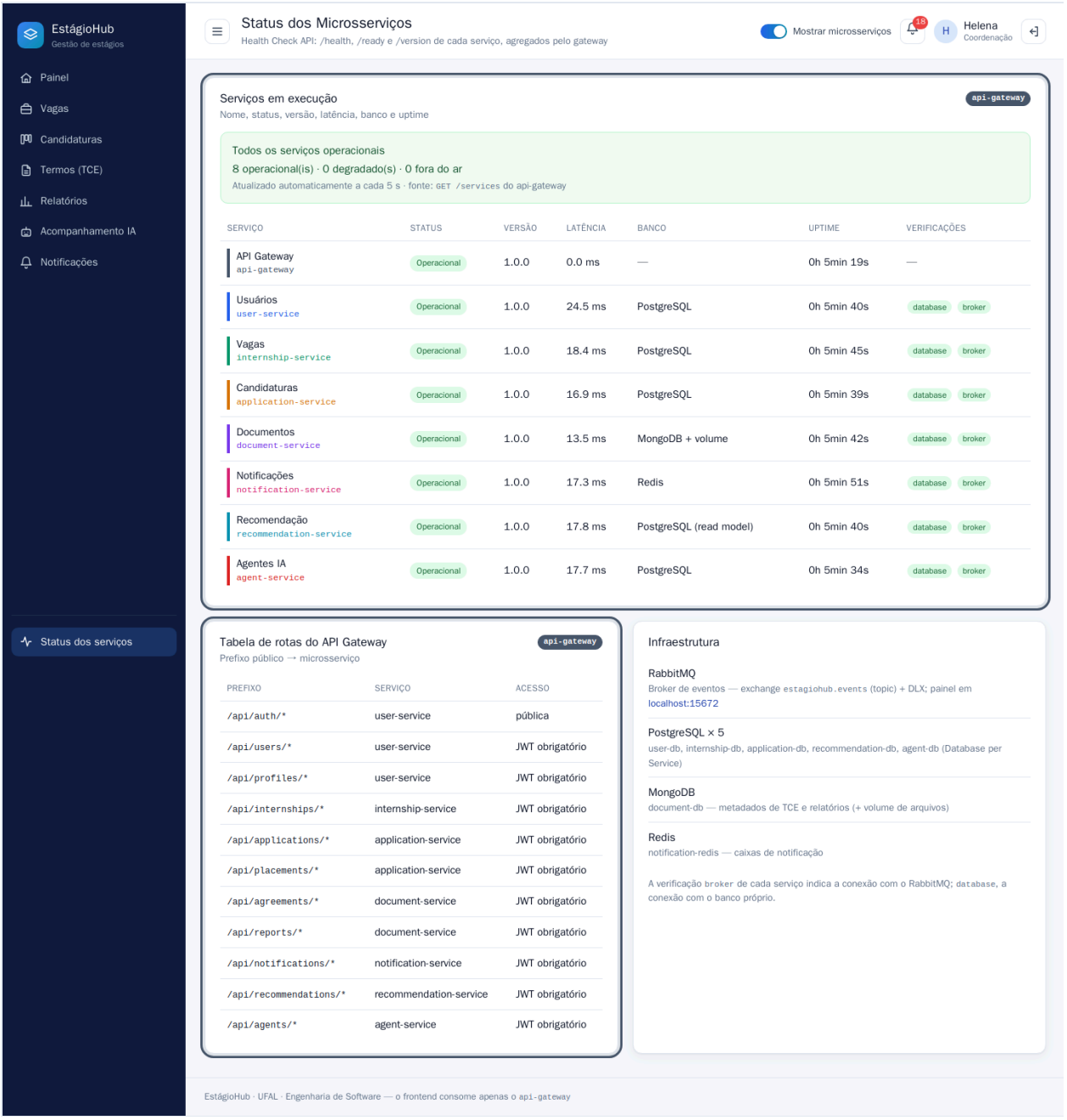


Figura 7 — Tela de status dos microsserviços (api-gateway).

Saída de `docker compose ps` no momento da captura das telas:

```
NAME                                IMAGE                                STATUS
estagiohub-agent-db-1               postgres:16-alpine                   Up 5 minutes (healthy)
estagiohub-agent-service-1          estagiohub/agent-service             Up 5 minutes (healthy)
estagiohub-api-gateway-1            estagiohub/api-gateway               Up 4 minutes (healthy)
estagiohub-application-db-1         postgres:16-alpine                   Up 5 minutes (healthy)
estagiohub-application-service-1    estagiohub/application-service       Up 5 minutes (healthy)
estagiohub-document-db-1            mongo:7.0                            Up 5 minutes (healthy)
estagiohub-document-service-1       estagiohub/document-service          Up 5 minutes (healthy)
estagiohub-internship-db-1          postgres:16-alpine                   Up 5 minutes (healthy)
estagiohub-internship-service-1     estagiohub/internship-service        Up 5 minutes (healthy)
estagiohub-notification-redis-1     redis:7.4-alpine                     Up 5 minutes (healthy)
estagiohub-notification-service-1   estagiohub/notification-service      Up 5 minutes (healthy)
estagiohub-rabbitmq-1               rabbitmq:3.13-management-alpine      Up 5 minutes (healthy)
estagiohub-recommendation-db-1      postgres:16-alpine                   Up 5 minutes (healthy)
estagiohub-recommendation-service-1 estagiohub/recommendation-service   Up 5 minutes (healthy)
estagiohub-user-db-1                postgres:16-alpine                   Up 5 minutes (healthy)
estagiohub-user-service-1           estagiohub/user-service              Up 5 minutes (healthy)
estagiohub-web-frontend-1           estagiohub/web-frontend              Up 4 minutes (healthy)
```

[PREENCHER: print do terminal com `make up` e `docker compose ps`]

Parte 2 — Implementação

7 Tecnologias

7.1 Microsserviços

Tabela 25 — Microsserviços, linguagem, framework, banco e porta.

Microsserviço	Linguagem	Framework	Banco	Porta
api-gateway	Python 3.12	FastAPI + httpx	—	8000 (exposta)
user-service	Python 3.12	FastAPI + SQLAlchemy 2	PostgreSQL 16	8000 (interna)
internship-service	Python 3.12	FastAPI + SQLAlchemy 2	PostgreSQL 16	8000 (interna)
application-service	Python 3.12	FastAPI + SQLAlchemy 2	PostgreSQL 16	8000 (interna)
document-service	Python 3.12	FastAPI + ReportLab + pypdfium2	MongoDB 7.0 + volume	8000 (interna)
notification-service	Python 3.12	FastAPI	Redis 7.4	8000 (interna)
recommendation-service	Python 3.12	FastAPI + scikit-learn	PostgreSQL 16	8000 (interna)
agent-service	Python 3.12	FastAPI + Anthropic SDK + APScheduler	PostgreSQL 16	8000 (interna)
web-frontend	Python 3.12	FastAPI + Jinja2 + HTMX 2.0.4	—	8080 (exposta)
rabbitmq	—	RabbitMQ 3.13 (management)	—	15672 (painel)

7.2 Infraestrutura e bibliotecas

Tabela 26 — Infraestrutura e versões.

Item	Versão
Docker / Docker Compose (host de desenvolvimento)	Docker 29.8.0 / Compose v5.5.1 (API Compose v2)
Imagens de banco e *broker*	`postgres:16-alpine`, `mongo:7.0`, `redis:7.4-alpine`, `rabbitmq:3.13-management-alpine`
Imagem base dos serviços	`python:3.12-slim` (multi-stage, usuário não-root uid 10001)
FastAPI / uvicorn / pydantic	0.115.6 / 0.34.0 / 2.10.4
SQLAlchemy / asyncpg / Alembic	2.0.36 / 0.30.0 / 1.14.0
aio-pika / httpx	9.5.4 / 0.28.1
PyJWT / argon2-cffi	2.10.1 / 23.1.0
PyMongo (API assíncrona) / redis-py	4.10.1 / 5.2.1
ReportLab / pypdfium2 / Pillow	4.2.5 / 4.30.0 / 11.0.0
scikit-learn / NumPy	1.6.0 / 2.2.1
anthropic (SDK) / APScheduler	`>=0.40,<2` / 3.11.0

Item	Versão
Jinja2 / HTMX	3.1.5 / 2.0.4 (empacotado localmente)
pytest / pytest-asyncio / pytest-cov / ruff	8.3.4 / 0.25.0 / 6.0.0 / 0.8.4
Faker (dados sintéticos)	33.1.0
Pandoc / Mermaid CLI / Playwright	3.6 / 11.4.2 / 1.49.1 (todos em containers)
CI	GitHub Actions (`.github/workflows/ci.yml`)

8 Funcionamento do sistema

Os prints a seguir foram gerados automaticamente por `make screenshots` (`scripts/capture_screenshots.py`, Playwright), que **executa o fluxo principal pela própria interface** — preenchendo formulários e clicando nos botões — na stack do Compose com os dados do seed. Salvo indicação, o *toggle* “Mostrar microsserviços” está ligado: cada painel aparece contornado com a cor do serviço que forneceu seus dados (header `X-Served-By` repassado pelo gateway). O mesmo fluxo é verificado por `make e2e` diretamente na API.

Passo 1 — Login. Todos os papéis entram pela mesma tela; o `user-service` valida a senha (argon2) e emite o JWT, guardado em *cookie* HttpOnly.

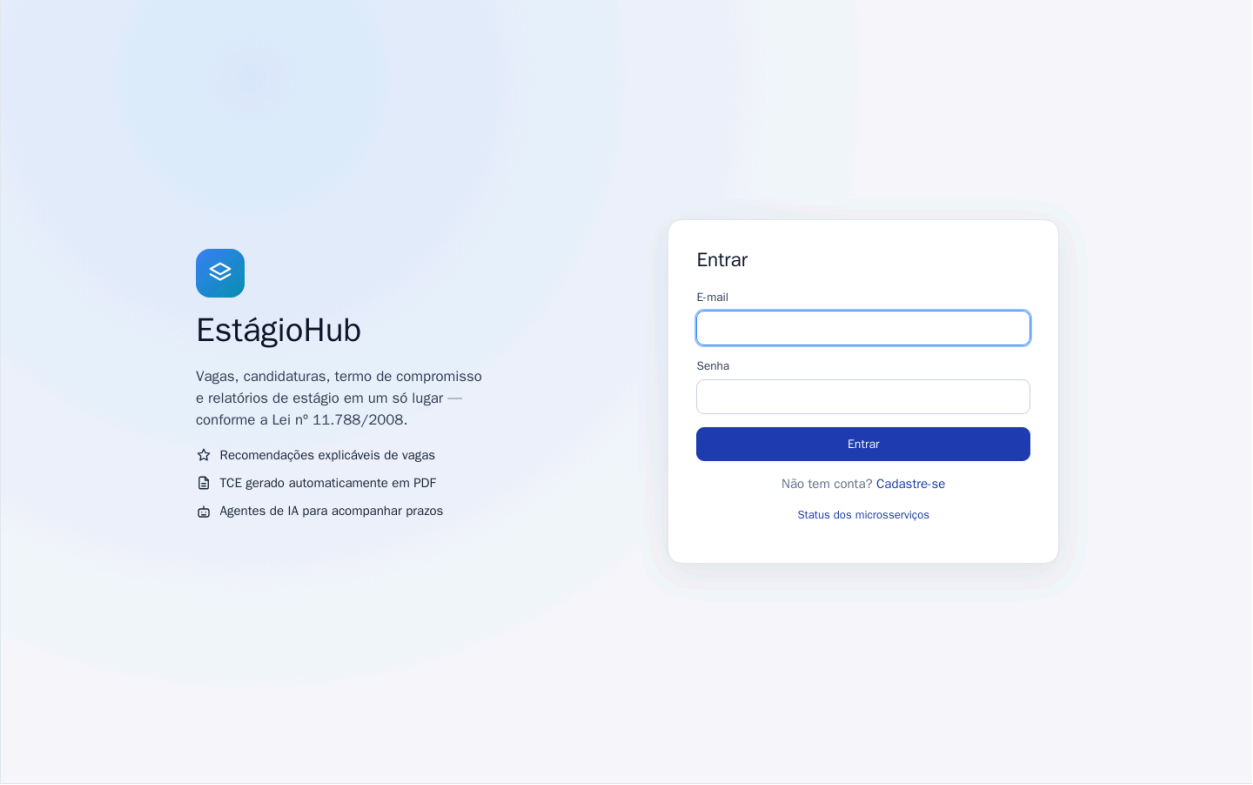


Figura 8 — Login. Serviços: web-frontend → api-gateway → user-service.

Passo 2 — Dashboard da empresa com a legenda dos microsserviços desta tela.

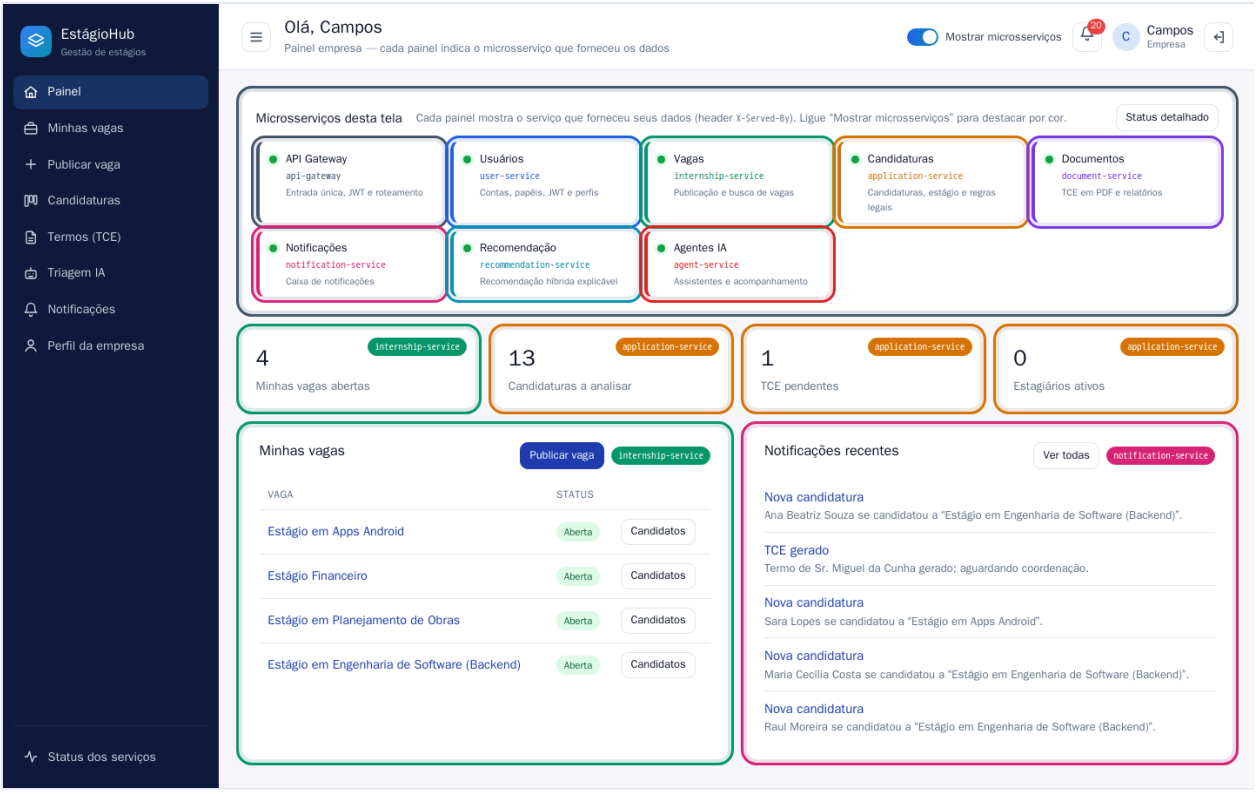


Figura 9 — Dashboard da empresa. Serviços: internship-service, application-service, notification-service; legenda via api-gateway (`GET /services`).

Passo 3 — A empresa publica uma vaga com requisitos, curso, período mínimo, carga horária, modalidade e bolsa. A jornada é validada no backend.

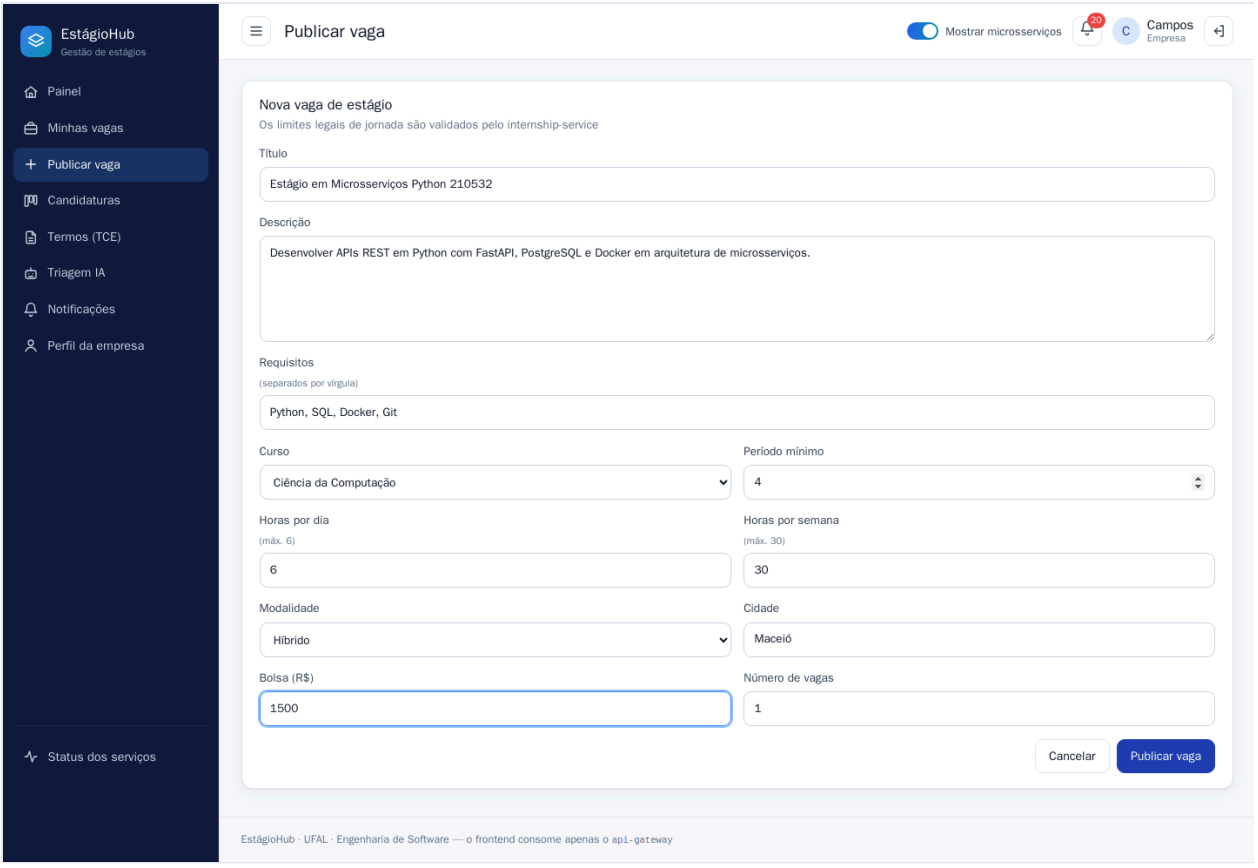


Figura 10 — Publicação de vaga. Serviço: internship-service (publica `internship.published`).

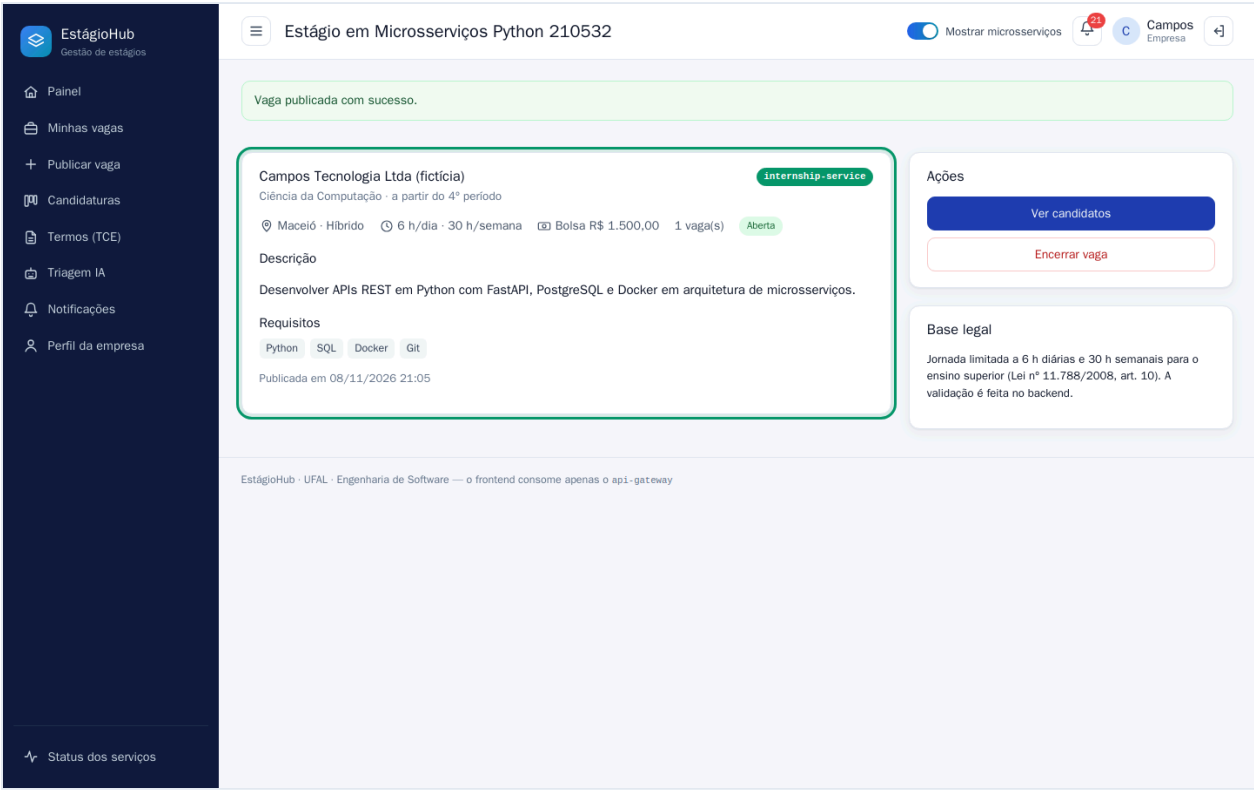


Figura 11 — Vaga publicada. Serviço: internship-service.

Passo 4 — O estudante vê as vagas recomendadas com a explicação do porquê. O evento `internship.published` alimentou o modelo de leitura do `recommendation-service`.

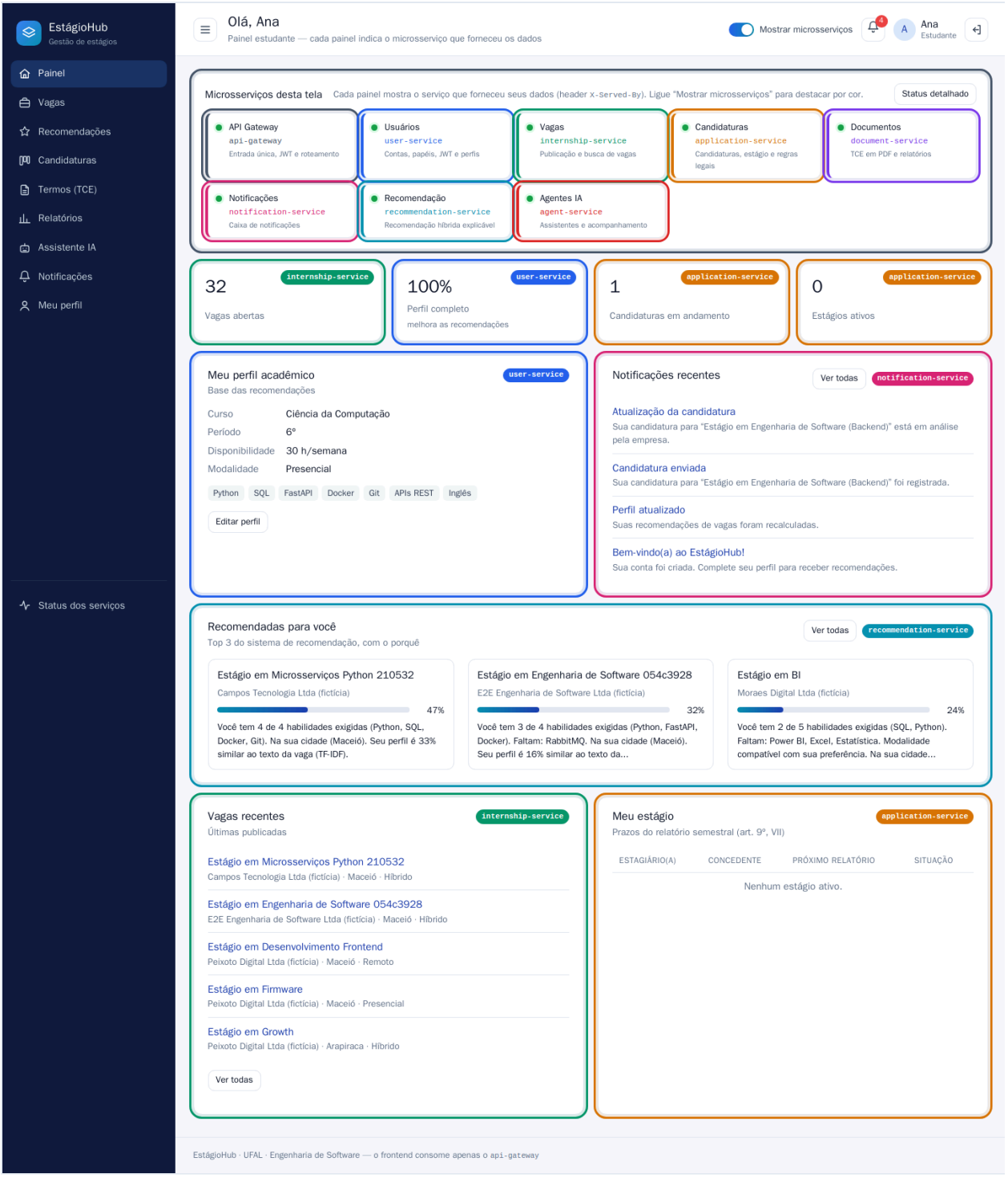


Figura 12 — Dashboard do estudante com o top 3 recomendado. Serviços: user-service, internship-service, application-service, recommendation-service, notification-service.

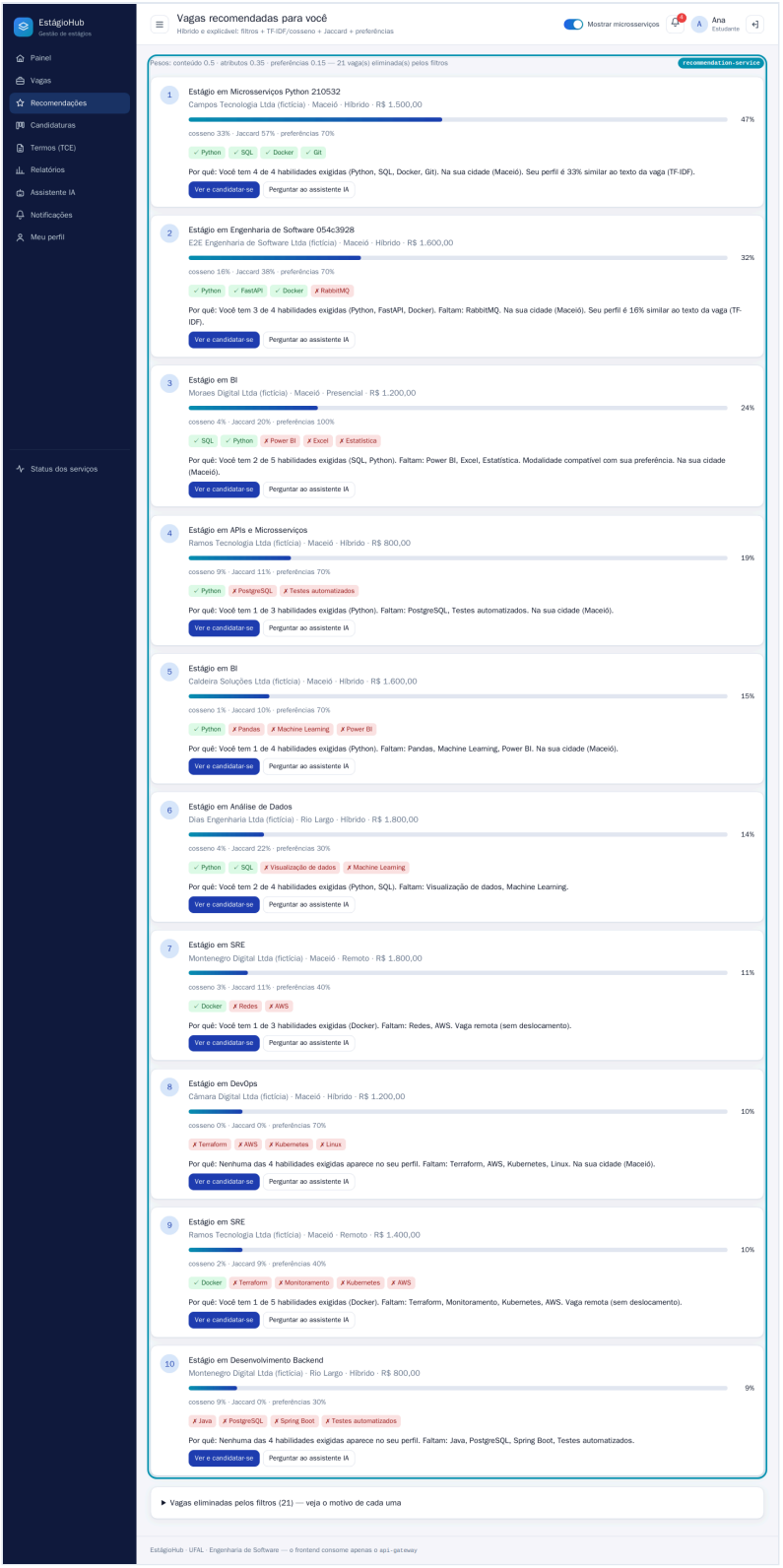


Figura 13 — Recomendações explicadas (habilidades em comum ✓, faltantes ✗, componentes do *score*). Serviço: recommendation-service.

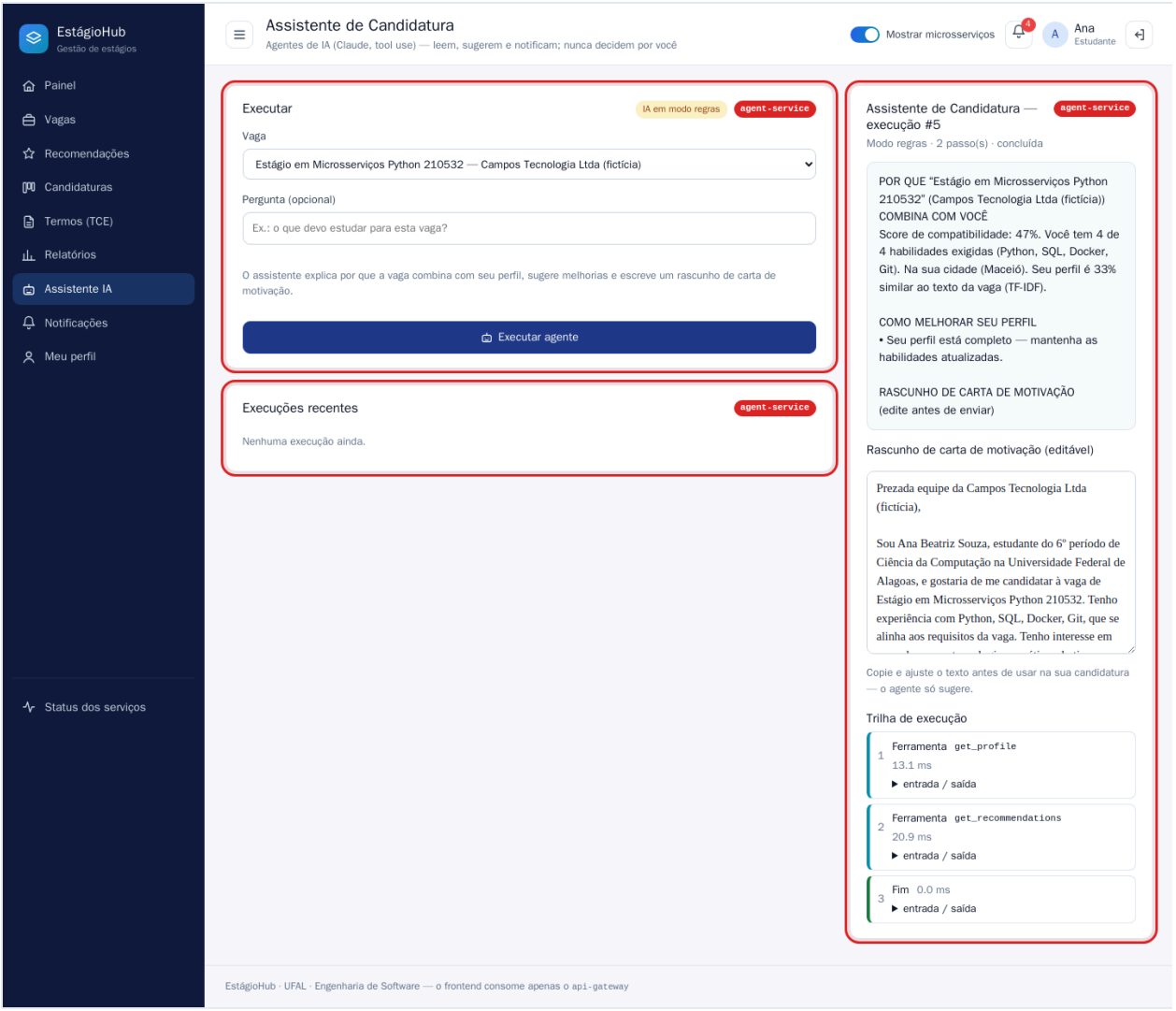


Figura 14 — Assistente de Candidatura com a trilha de execução (ferramentas `get_profile` e `get_recommendations`) e rascunho de carta editável. Serviços: agent-service → user-service, recommendation-service.

Passo 5 — O estudante se candidata (publica `application.submitted`).

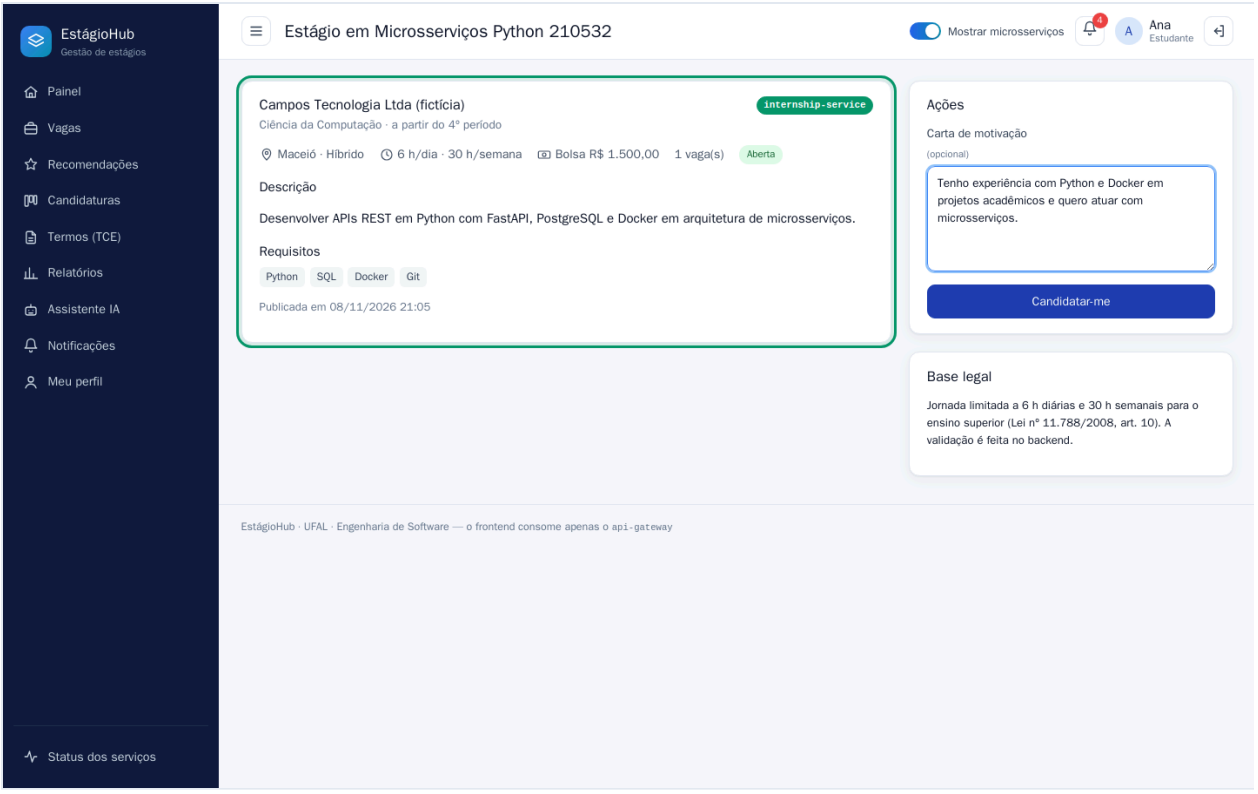


Figura 15 — Candidatura com carta de motivação. Serviços: internship-service (vaga), application-service (candidatura).

Passo 6 — A empresa vê o ranking de candidatos e aprova um deles.

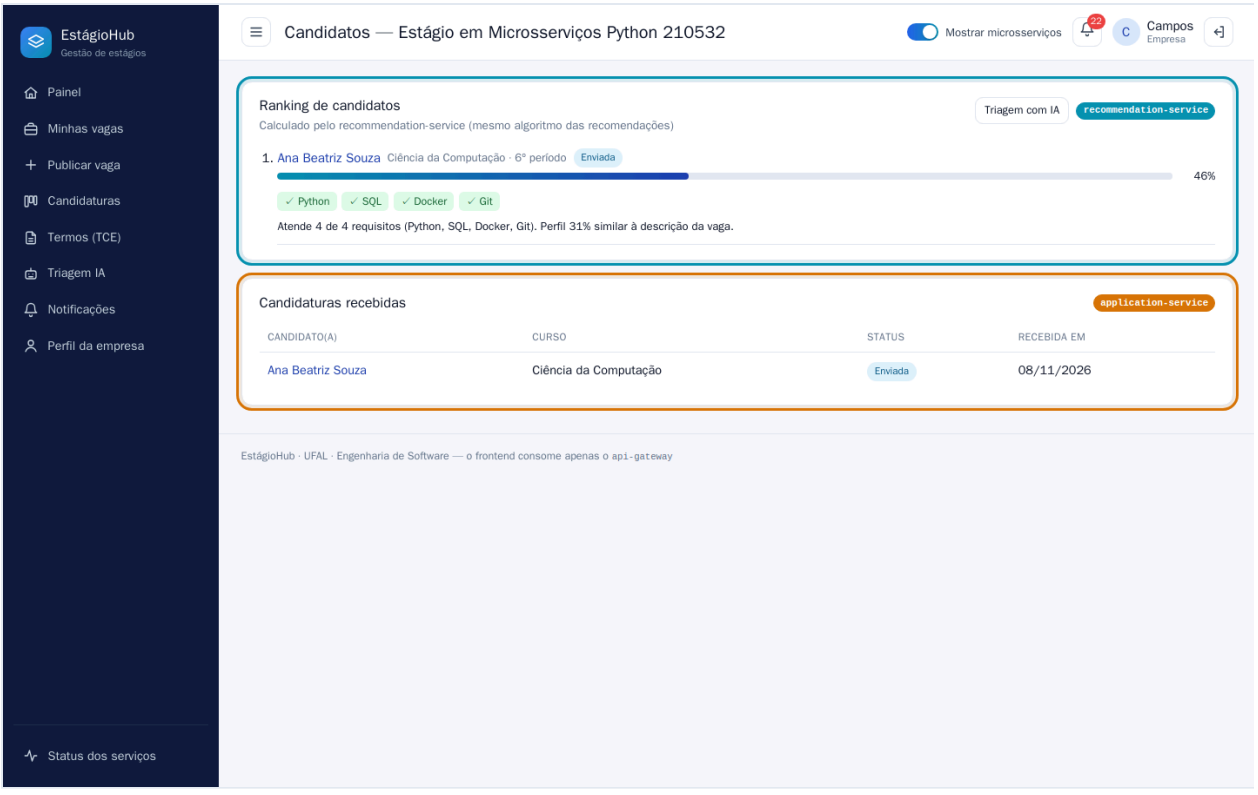


Figura 16 — Ranking de candidatos da vaga. Serviços: recommendation-service (ranking), application-service (candidaturas).

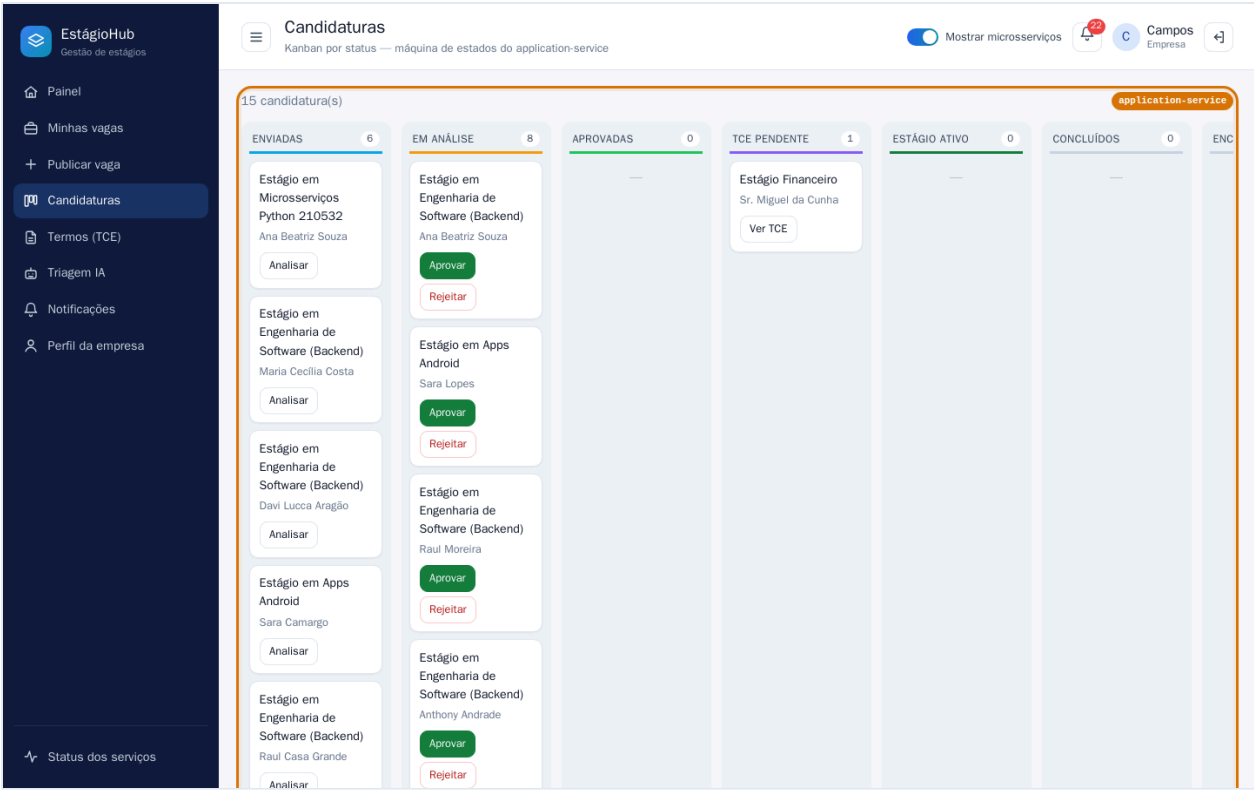


Figura 17 — Kanban de candidaturas da empresa. Serviço: application-service.

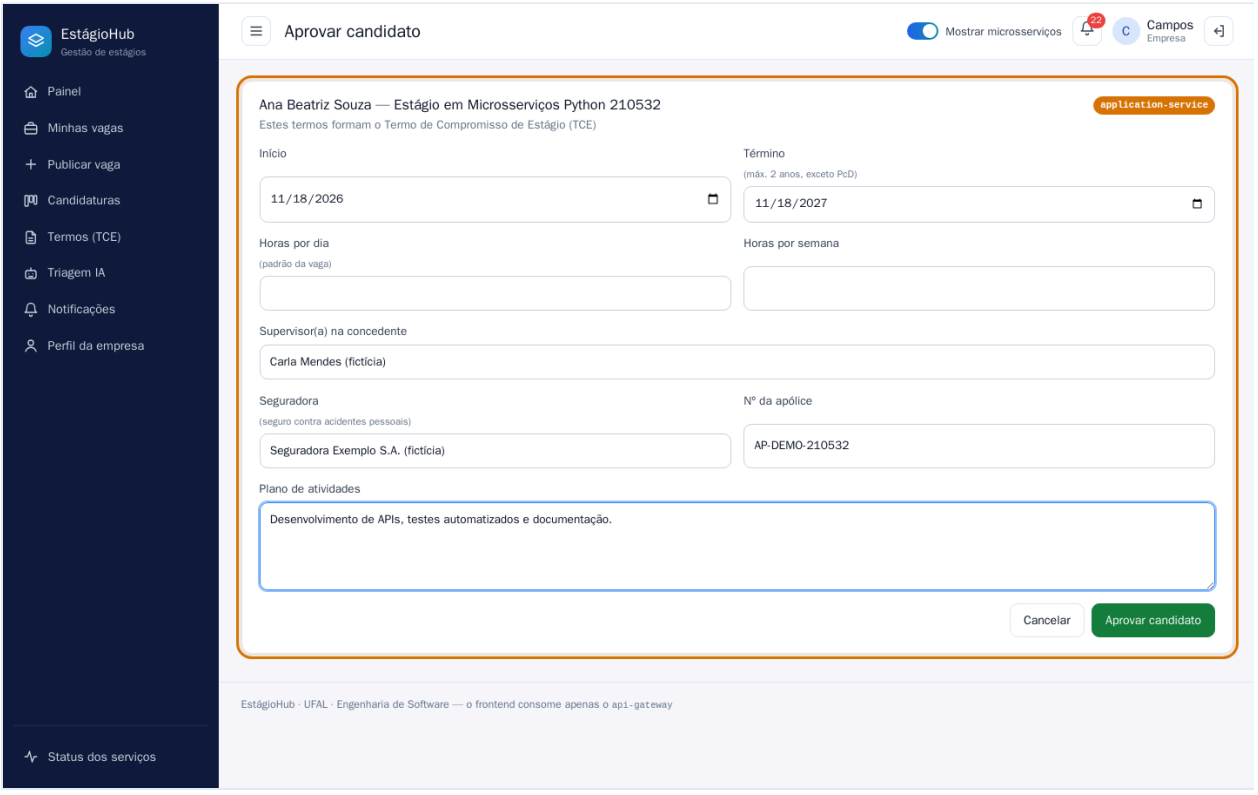


Figura 18 — Aprovação com os termos do estágio e o seguro (regras da Lei 11.788 validadas pelo application-service).

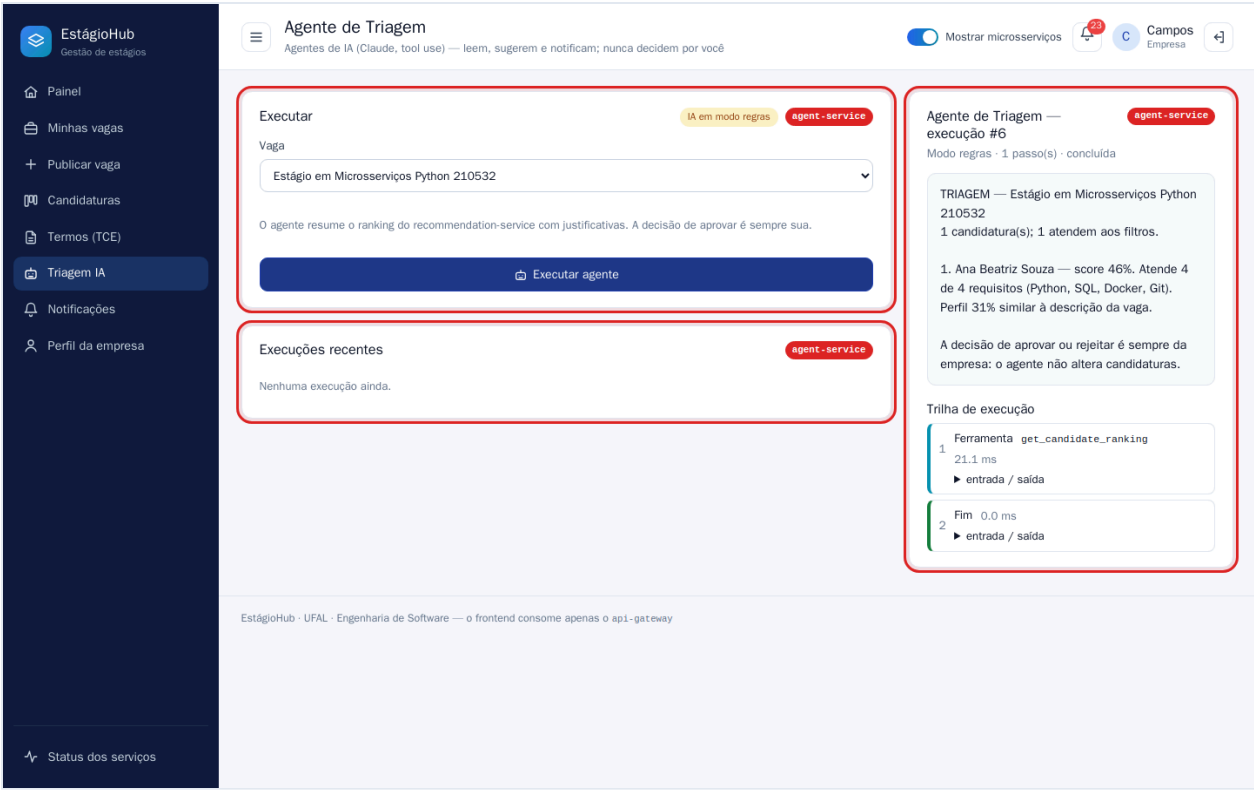


Figura 19 — Agente de Triagem resumindo o ranking. Serviços: agent-service → recommendation-service.

Passo 7 — O TCE é gerado automaticamente em PDF (saga: o `document-service` reage a `application.status_changed` e publica `agreement.generated`; o `application-service` move a candidatura para `agreement_pending`).

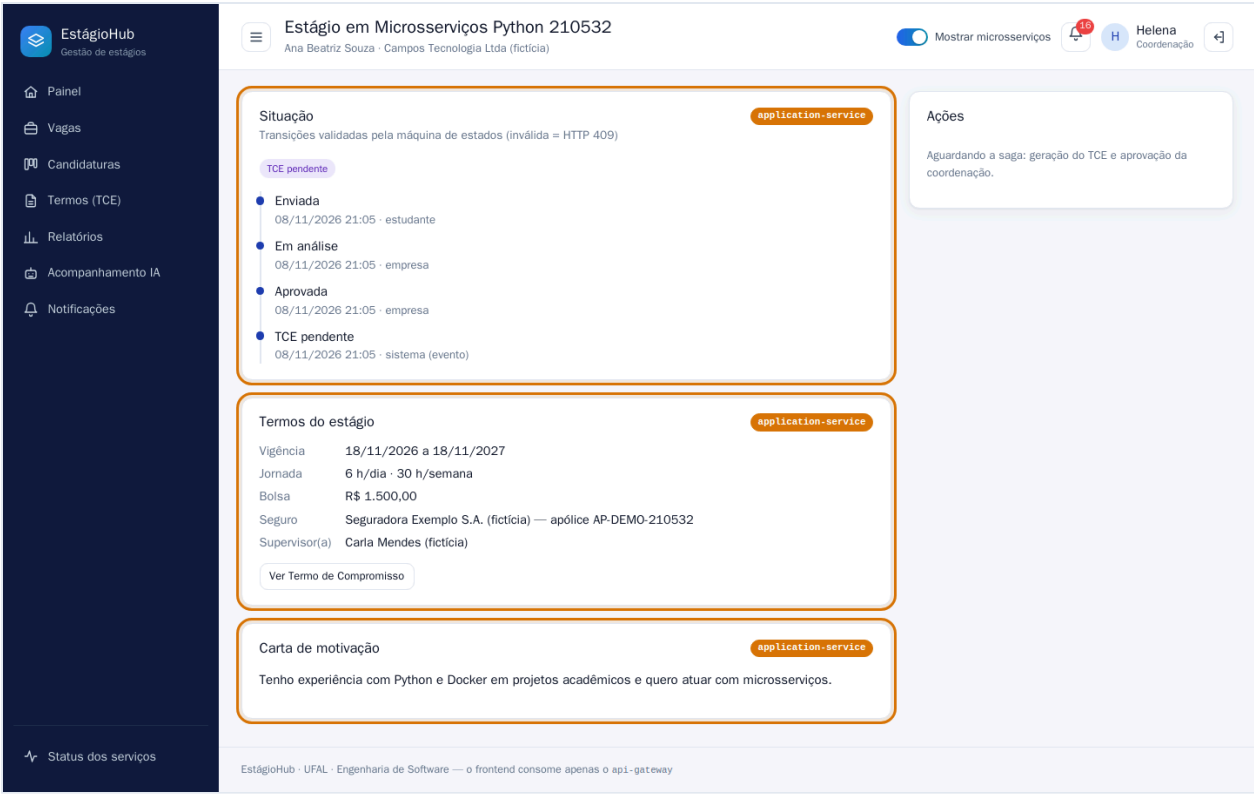


Figura 20 — Candidatura em “TCE pendente”, com o histórico das transições (inclusive as feitas pelo sistema via eventos). Serviço: application-service.

Passo 8 — A coordenação aprova o termo e o estágio passa a “ativo”.

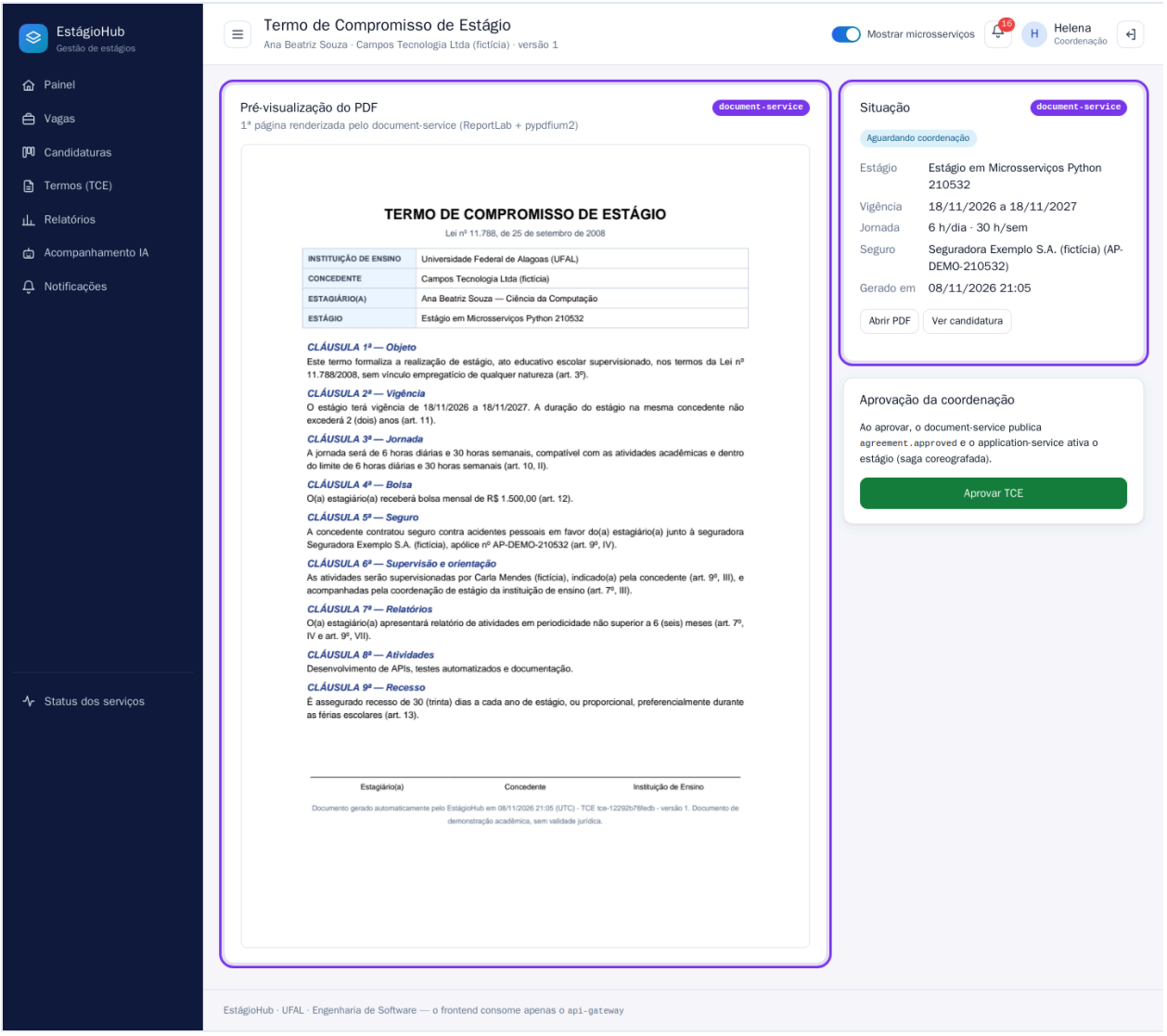


Figura 21 — Pré-visualização do TCE e aprovação pela coordenação. Serviço: document-service (PDF, *preview* e `agreement.approved`).

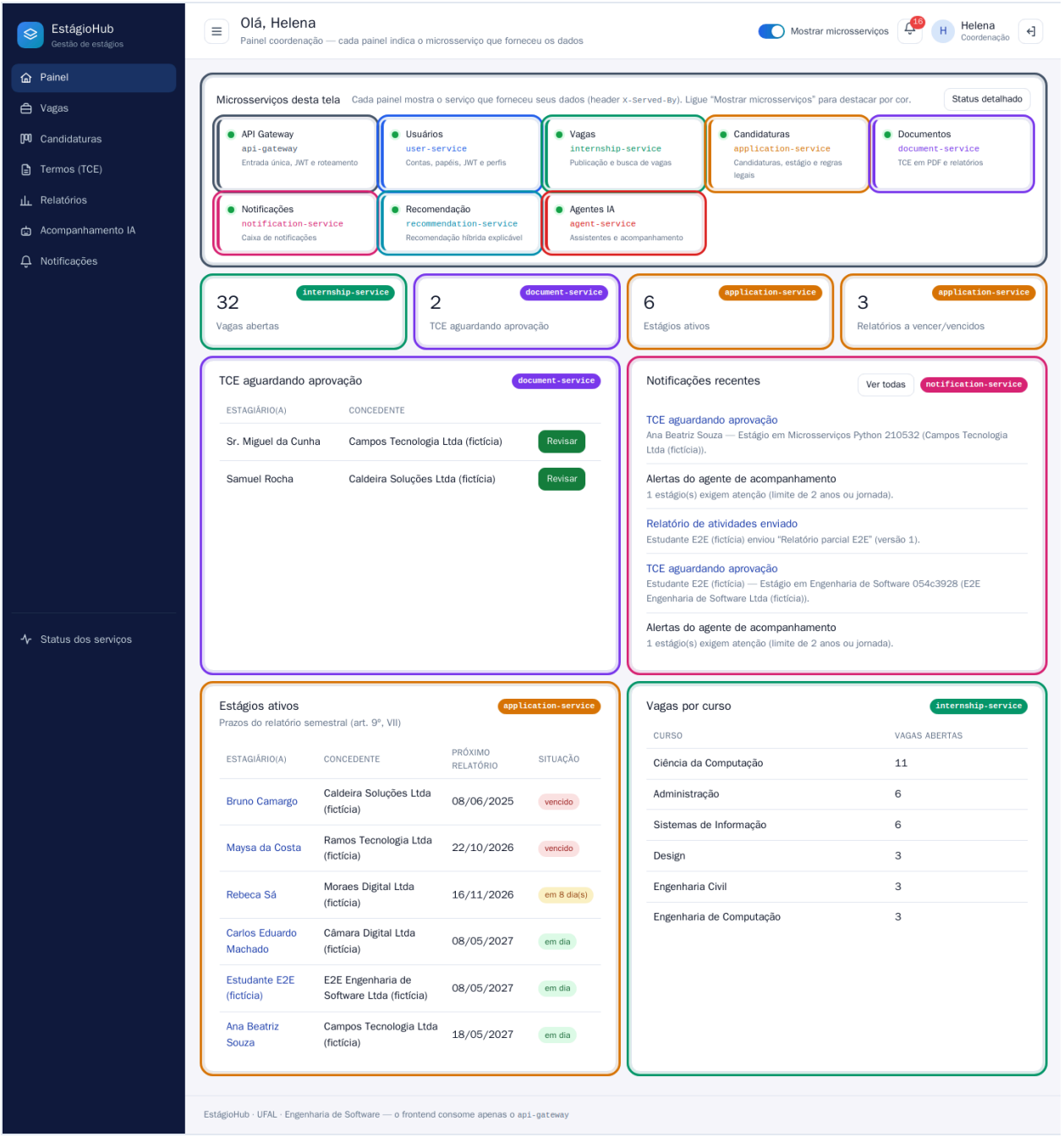


Figura 22 — Dashboard da coordenação: TCEs pendentes, estágios ativos com prazos de relatório e notificações. Serviços: internship-service, document-service, application-service, notification-service.

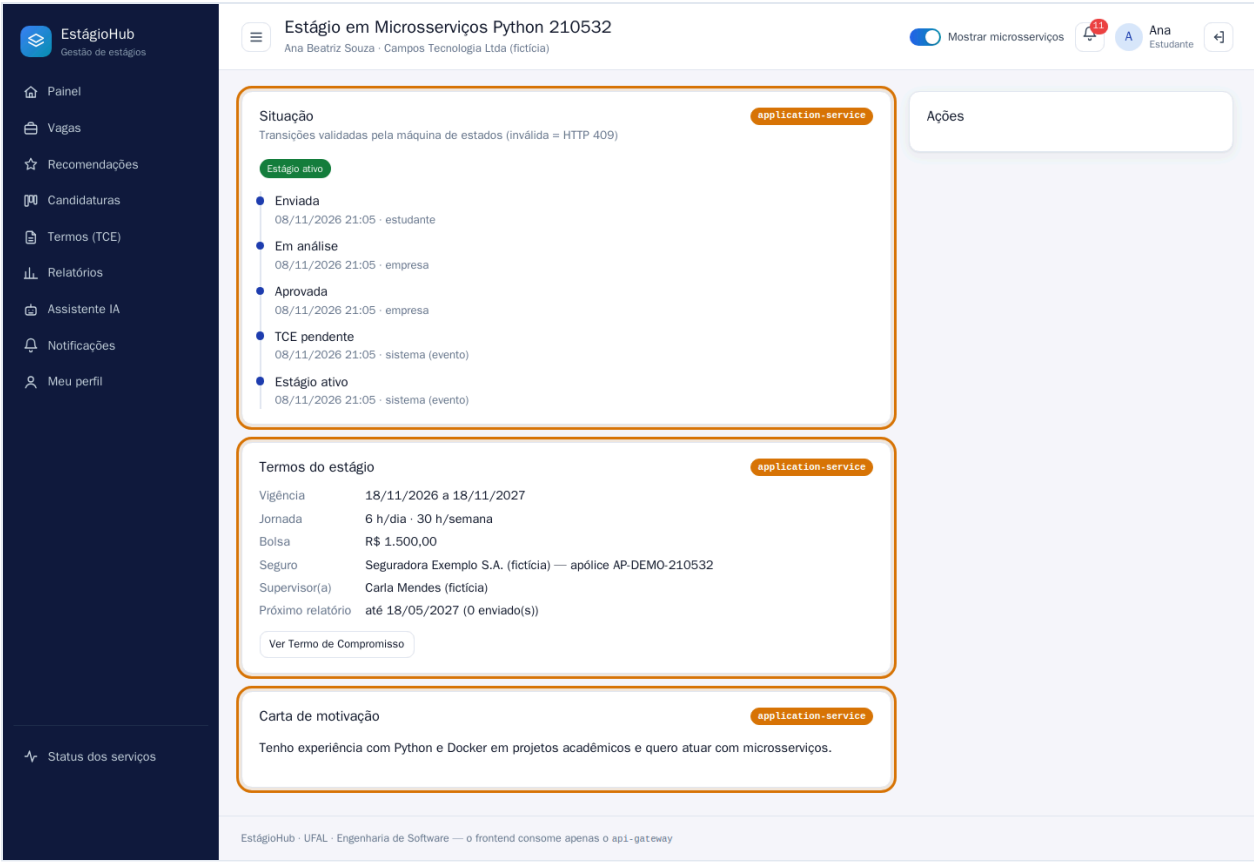


Figura 23 — Estágio ativo com o histórico completo da saga. Serviço: application-service.

Passo 9 — O estudante envia o relatório de atividades e o agente de acompanhamento alerta sobre prazos (`report.due_soon`).

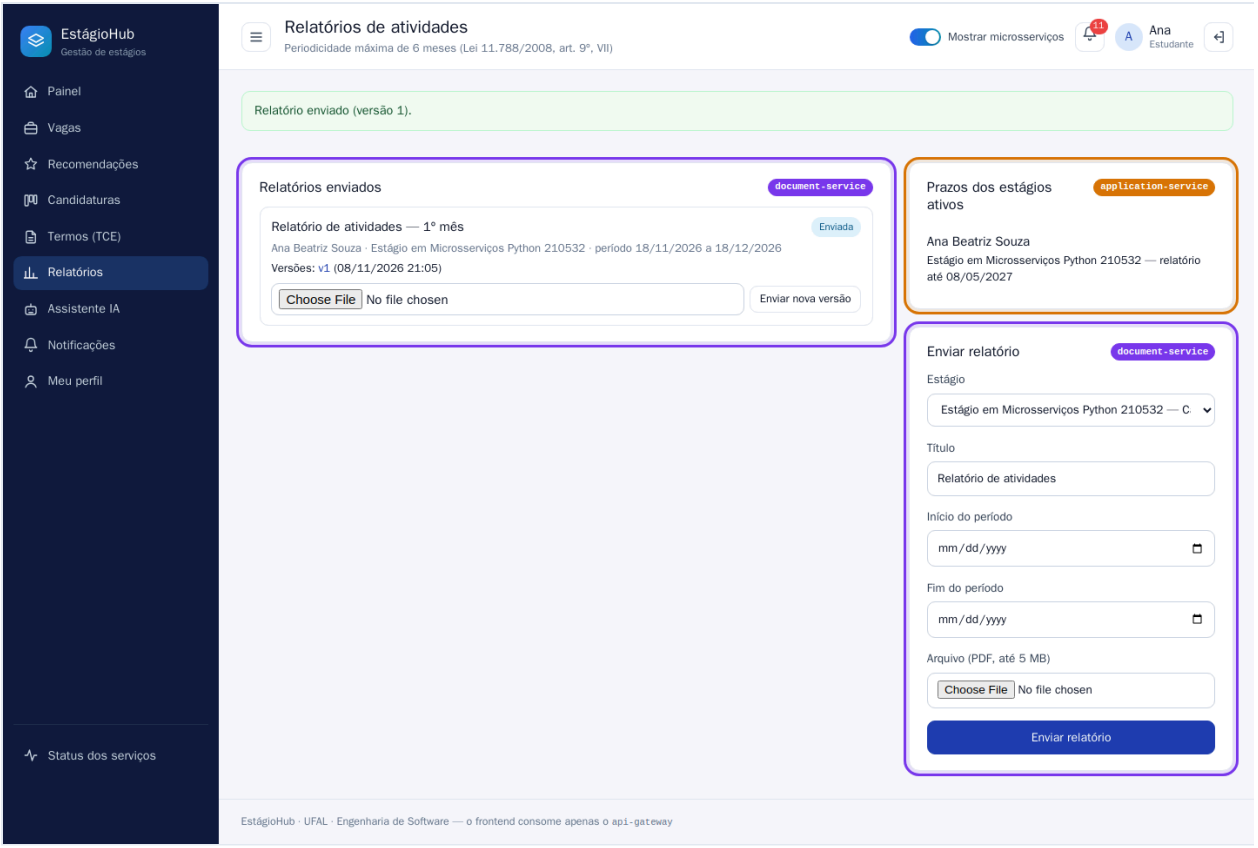


Figura 24 — Relatório enviado (versão 1) e prazos do estágio. Serviços: document-service (relatórios), application-service (prazos).

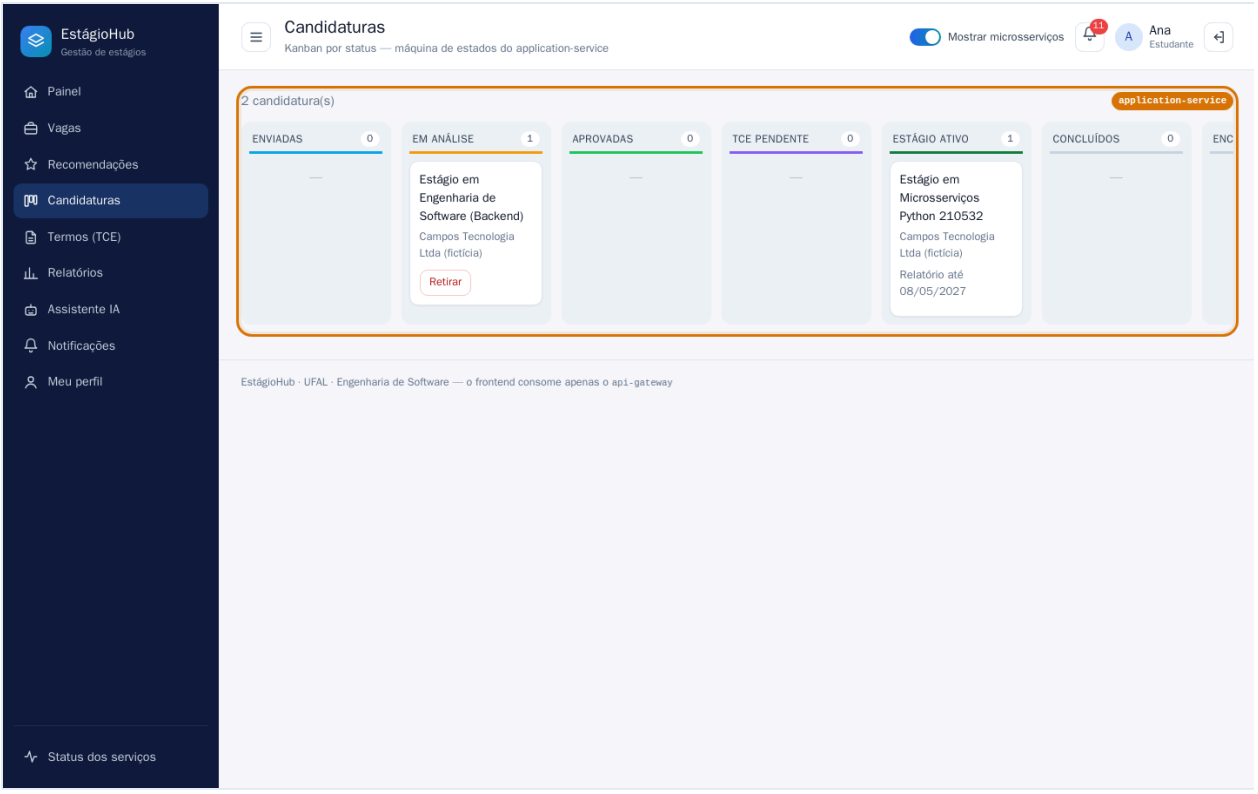


Figura 25 — Kanban do estudante. Serviço: application-service.

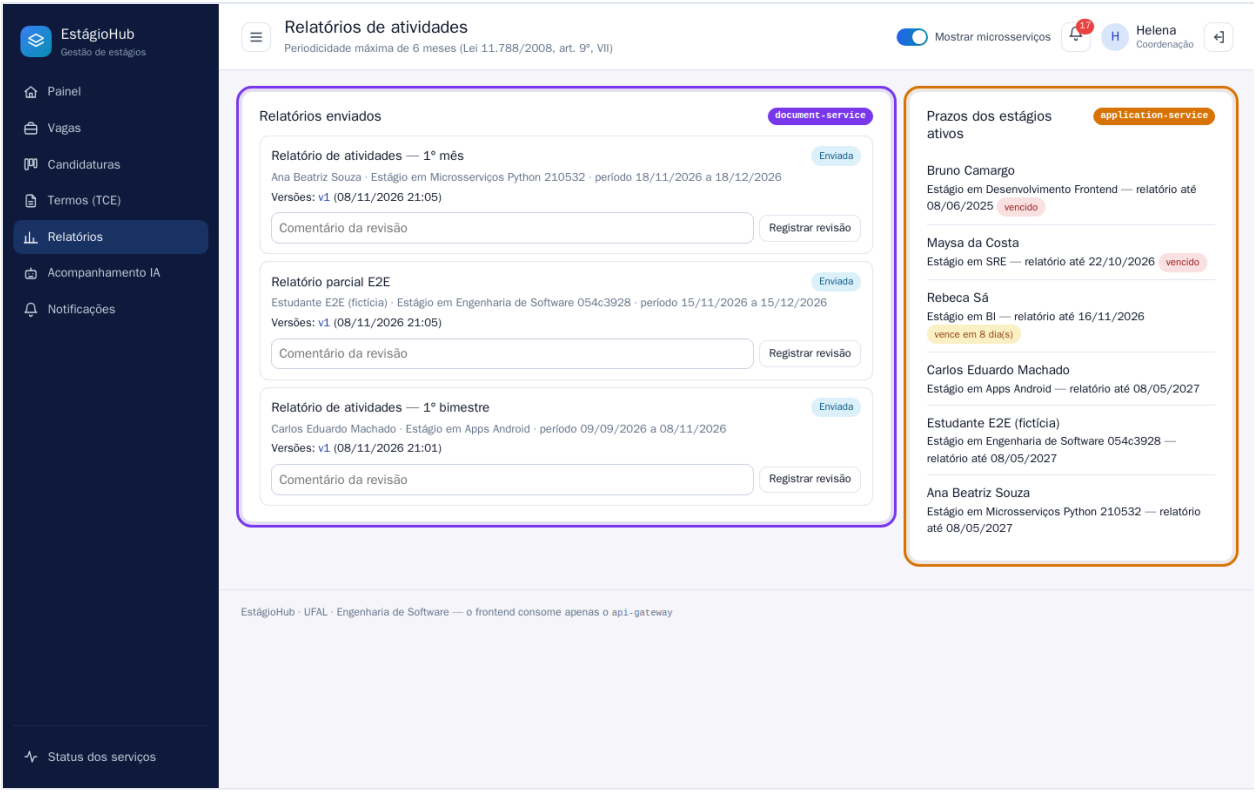


Figura 26 — Relatórios e prazos vistos pela coordenação (vencidos e a vencer). Serviços: document-service, application-service.

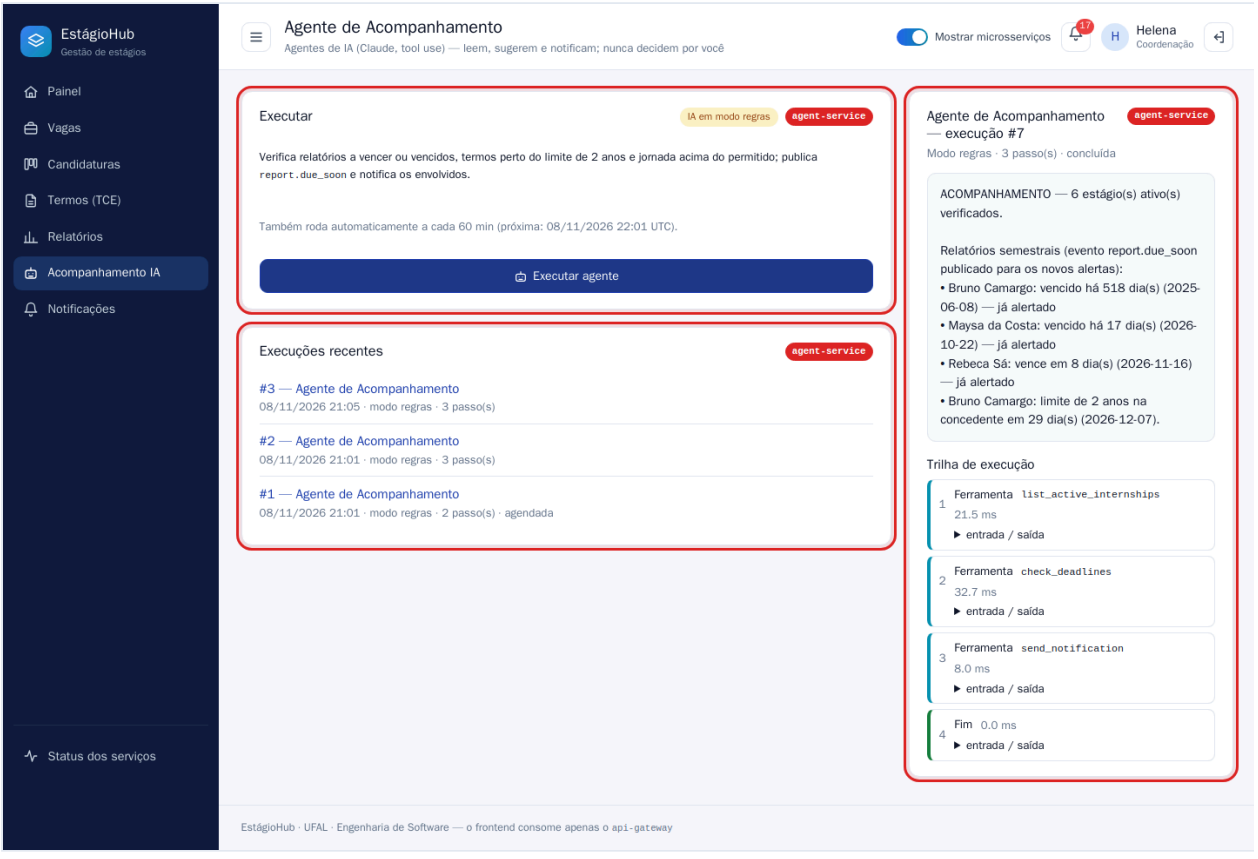


Figura 27 — Agente de Acompanhamento: relatórios vencidos/a vencer, limite de 2 anos, notificação à coordenação e trilha. Serviços: agent-service → application-service, notification-service.

Passo 10 — Cada transição gerou notificações para os atores envolvidos.

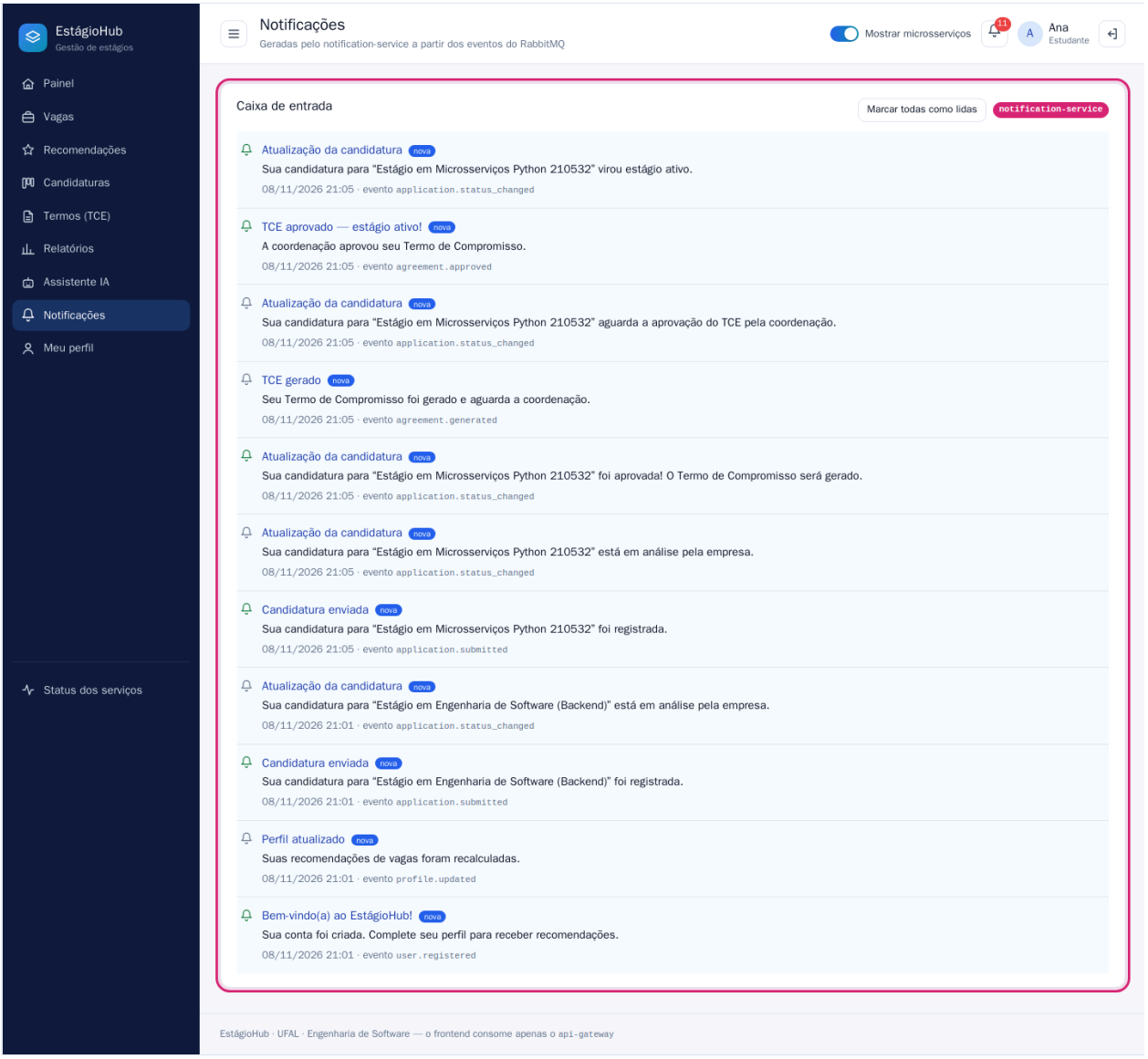


Figura 28 — Notificações do estudante, cada uma com o evento de origem. Serviço: notification-service.

Microsserviços em destaque e degradação graciosa. A Figura 29 mostra a mesma tela da Figura 9 com o *toggle* desligado (apenas os *badges* discretos). As Figuras 30 a 32 foram capturadas com o `recommendation-service` e o `agent-service` parados: o restante do sistema segue funcionando e a interface avisa qual serviço está indisponível.

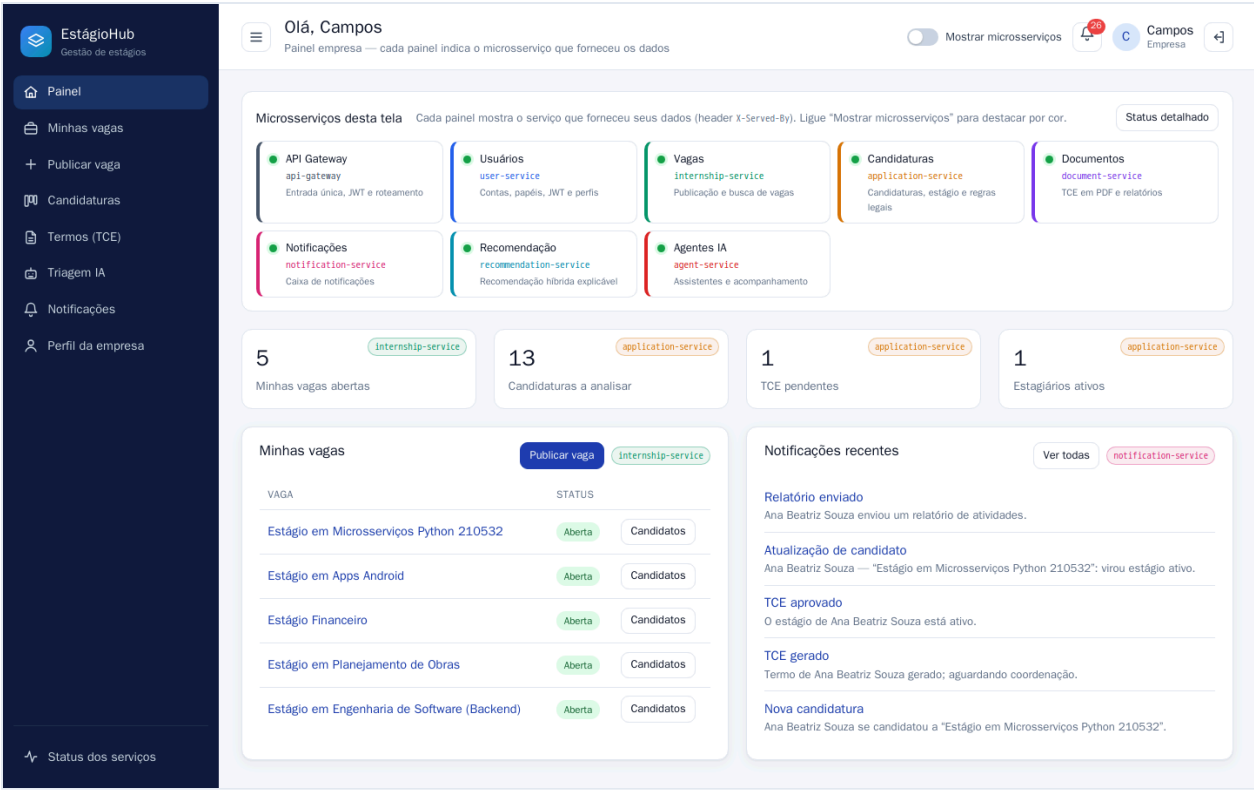


Figura 29 — Dashboard com o *toggle* "Mostrar microsserviços" desligado.

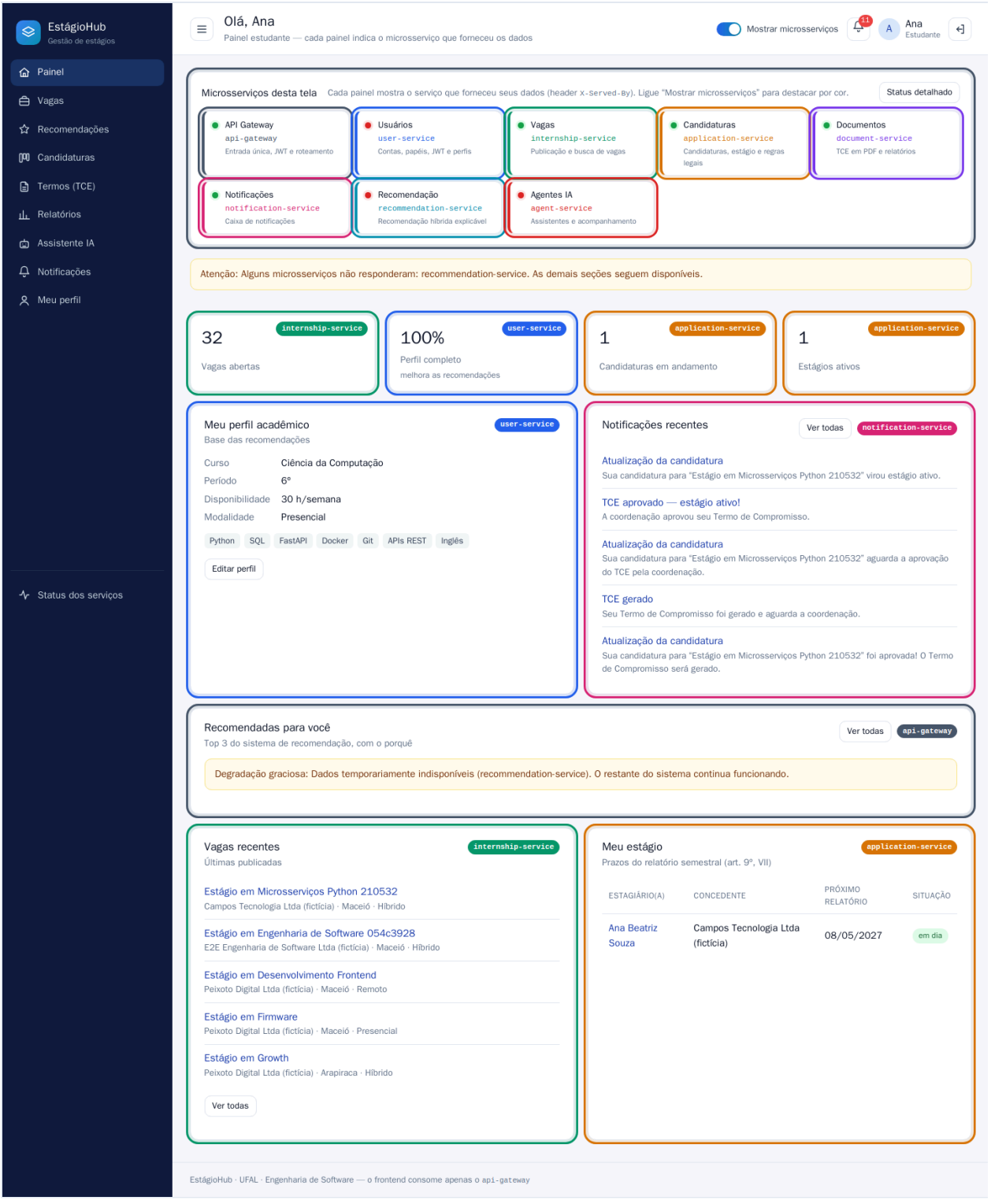


Figura 30 — Degradação graciosa no dashboard do estudante (recommendation-service parado).

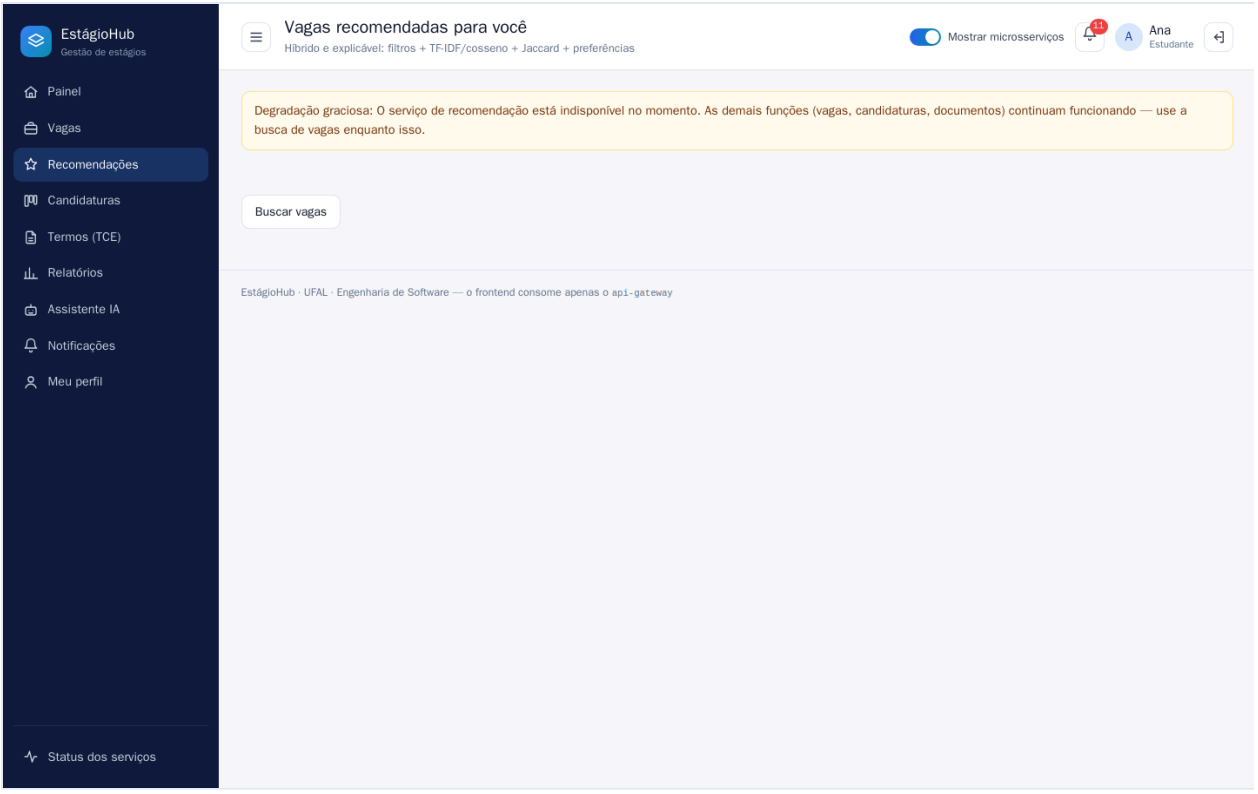


Figura 31 — Página de recomendações com o serviço indisponível.

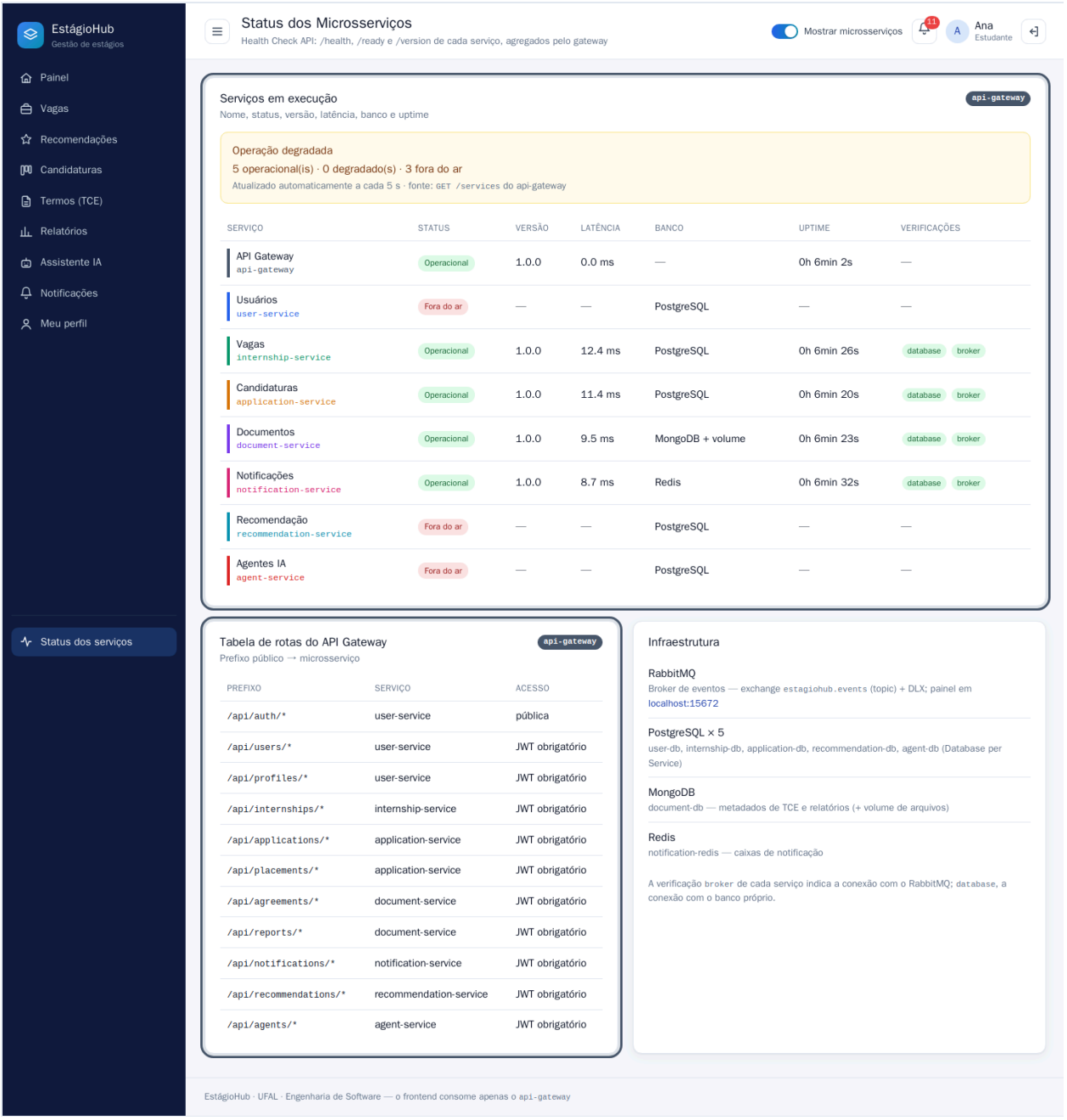


Figura 32 — Status com dois serviços fora do ar (circuit breaker do gateway e verificação de saúde).

9 Repositório

- **Link:** https://github.com/victorMAOliveira/projeto-eng-soft
- **Pré-requisitos:** Docker com Compose v2, Git e GNU Make. Testes, seed, prints e relatório rodam em containers.
- **Executar em três comandos:**

```
make env    # cria .env a partir de .env.example e gera os segredos
make up     # constrói e sobe os 17 containers e espera todos ficarem healthy
make seed   # carrega os dados de demonstração (sintéticos) via gateway
```

- **Acessos:** interface em http://localhost:8080, gateway em http://localhost:8000 (`/docs` de cada serviço via `docker compose exec`; `GET /services`), painel do RabbitMQ em http://localhost:15672.
- **Contas de demonstração (fictícias):** `coordenacao@ufal.test` / `Coord@2026`; `rh@techsolucoes.test` / `Empresa@2026`; `estudante@ufal.test` / `Estudante@2026`; `calouro@ufal.test` / `Estudante@2026` (perfil vazio, *cold start*).
- **Demais alvos:** `make test`, `make e2e`, `make eval`, `make screenshots`, `make report`, `make all`; `make clean` remove containers e volumes.
- **Agentes com IA real:** preencher `ANTHROPIC_API_KEY` no `.env`; sem ela, os agentes funcionam em modo regras.
- **Estrutura:** `components/` (componentes), `services/` (microsserviços), `frontend/web/`, `scripts/`, `docs/` (arquitetura, ADRs, Scrum, relatório, guia de estudo).

10 Referências

BRASIL. **Lei nº 11.788, de 25 de setembro de 2008**. Dispõe sobre o estágio de estudantes. Diário Oficial da União, Brasília, DF, 26 set. 2008. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2008/lei/l11788.htm. Acesso em: 8 nov. 2026.

FOWLER, Martin. **CQRS**. 2011. Disponível em: https://martinfowler.com/bliki/CQRS.html. Acesso em: 8 nov. 2026.

GARCIA-MOLINA, Hector; SALEM, Kenneth. Sagas. In: ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA, 1987, San Francisco. **Proceedings** […]. New York: ACM, 1987. p. 249–259.

HOHPE, Gregor; WOOLF, Bobby. **Enterprise integration patterns**: designing, building, and deploying messaging solutions. Boston: Addison-Wesley, 2003.

JÄRVELIN, Kalervo; KEKÄLÄINEN, Jaana. Cumulated gain-based evaluation of IR techniques. **ACM Transactions on Information Systems**, v. 20, n. 4, p. 422–446, 2002.

LEWIS, James; FOWLER, Martin. **Microservices**: a definition of this new architectural term. 2014. Disponível em: https://martinfowler.com/articles/microservices.html. Acesso em: 8 nov. 2026.

MANNING, Christopher D.; RAGHAVAN, Prabhakar; SCHÜTZE, Hinrich. **Introduction to information retrieval**. Cambridge: Cambridge University Press, 2008.

NEWMAN, Sam. **Building microservices**: designing fine-grained systems. 2. ed. Sebastopol: O'Reilly Media, 2021.

NYGARD, Michael T. **Documenting architecture decisions**. 2011. Disponível em: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions. Acesso em: 8 nov. 2026.

NYGARD, Michael T. **Release it!**: design and deploy production-ready software. 2. ed. Raleigh: Pragmatic Bookshelf, 2018.

PEDREGOSA, Fabian et al. Scikit-learn: machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

RICHARDSON, Chris. **Microservices patterns**: with examples in Java. Shelter Island: Manning, 2018.

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum Guide**: the definitive guide to Scrum — the rules of the game. 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 8 nov. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

SZYPERSKI, Clemens; GRUNTZ, Dominik; MURER, Stephan. **Component software**: beyond object-oriented programming. 2. ed. London: Addison-Wesley, 2002.

Documentações oficiais consultadas (acesso em: 8 nov. 2026): FastAPI (https://fastapi.tiangolo.com/); SQLAlchemy (https://docs.sqlalchemy.org/); Alembic (https://alembic.sqlalchemy.org/); RabbitMQ (https://www.rabbitmq.com/docs); aio-pika (https://docs.aio-pika.com/); Docker Compose (https://docs.docker.com/compose/); PostgreSQL (https://www.postgresql.org/docs/); MongoDB (https://www.mongodb.com/docs/); Redis (https://redis.io/docs/); scikit-learn (https://scikit-learn.org/stable/); HTMX (https://htmx.org/docs/); Anthropic — Claude API (https://docs.anthropic.com/); Pandoc (https://pandoc.org/MANUAL.html); Playwright (https://playwright.dev/python/); Mermaid (https://mermaid.js.org/).

11 Vídeo

Não se aplica (o sistema não é um jogo).

Apêndice A — Checklist de conformidade

Tabela 27 — Checklist de conformidade com o modelo de relatório.

Item do modelo	Seção	Evidência
Capa com UFAL, disciplina, professor, título, integrantes, link do GitHub, local/ano	Capa	Capa deste documento (link: https://github.com/victorMAOliveira/projeto-eng-soft)
Sumário	Sumário	Gerado automaticamente
1 Enunciado (problema, objetivo, escopo, atores, RF, RNF)	1	Tabelas 1–3
2 Metodologia (Scrum, justificativa, papéis, cerimônias, duração)	2	Tabelas 4–5; `docs/scrum/`
3 Técnicas de reuso (Microservices e Componentes de Software, definições)	3	Seções 3.1 e 3.2 com referências
4 Desenvolvimento (narrativa por sprint, aplicação das técnicas)	4.1	`docs/scrum/sprint-0..4.md`, tags `sprint-0..4`
4 Catálogo de componentes	4.2, 6.3.9	Tabelas 6 e 23; `components/*/README.md`
4 Matriz de reuso	4.3	Tabela 7, Figura 1; `scripts/reuse_metrics.py`
4 Métricas de reuso	4.4	Tabela 8
4 Padrões de microsserviços	4.6	Tabela 10
4 Resumo dos ADRs	4.7	Tabela 11; `docs/architecture/adr/`
5 Product Backlog e Sprint Backlogs	5	Tabelas 12–13; `docs/scrum/product-backlog.md`
6.1 Arquitetura: contêineres, fluxo de eventos, sequência da saga	6.1	Figuras 2–5; `docs/architecture/*.mmd`
6.2 API Gateway: responsabilidades, rotas, código documentado	6.2	Tabelas 14–15; trechos de `proxy.py`, `routes.py`, `health_aggregator.py`
6.3 Cada microsserviço e componente: responsabilidade, endpoints, eventos, dados, código, link	6.3	Tabelas 16–23 e trechos de código
6.4 Recomendação: arquitetura, algoritmo, fórmula, explicabilidade, cold start, métricas, agentes, código	6.4	Figura 6, Tabela 24, trecho de `recommender.py`
6.5 Serviços em execução: print do Status, `docker compose ps`, print do terminal	6.5	Figura 7, saída de `docker compose ps`; print do terminal: [PREENCHER]
7 Tecnologias (tabela de serviços e infraestrutura com versões)	7	Tabelas 25–26
8 Funcionamento (passo a passo com prints e serviços envolvidos, *toggle* ligado)	8	Figuras 8–32; `make screenshots`, `make e2e`

Item do modelo	Seção	Evidência
9 Repositório (link e como executar)	9	Link: https://github.com/victorMAOliveira/projeto-eng-soft; README.md
10 Referências (ABNT, obras reais)	10	Lista acima
11 Vídeo	11	“Não se aplica”